HookSuite

Herramienta de auditoría de seguridad web con Inteligencia Artificial

Módulo · Máster en Ciberseguridad & IA | Curso 2026

Práctica · Práctica 3 · Del prototipo al producto

Grupo · Feet Lovers · sin número asignado

EQUIPO DE DESARROLLO

P1 **Ivan Medina**

P2 **Macarena Rogerio**

P3 **Nacho García**

P4 **Carlos Bañuelos**

P5 **José María López**

ÁREAS DE TRABAJO

Asumidas en común y sin reparto por persona (apartado 10).

Área	Qué se hizo
Frontend	Pulido del Spider y del Repeater, revisión de pantallas, nueva pestaña Manual y retirada de Red
Backend	Alta y acceso por usuario, aislamiento de datos y bus interno de mensajes
IA	Integración del análisis de hallazgos con el panel y control del gasto
Seguridad	Cierre de la API, orígenes permitidos, canal autenticado, gestión de secretos y limpieza del historial
Infraestructura y despliegue	Contenedores, servidor público y despliegue reproducible
Pruebas y evidencias	Plan de pruebas, ejecución y capturas
Documentación	Memoria técnica y README

---

github.com/Feet-Lovers/Proyecto-Evolve

Entrega · 16 de octubre de 2026

Índice

1. Resumen ejecutivo 4
2. Punto de partida 4
   2.1. Qué nos encontramos 5
   2.2. La lección más cara de la Práctica 1 5
   2.3. Numeración de requisitos 5
   2.4. Clasificación inicial de requisitos 5
   2.5. Sobre el feedback de la Práctica 1 6
3. Requisitos 6
   3.1. Requisitos funcionales (RF) 6
   3.2. Requisitos no funcionales (RNF) 7
   3.3. Requisitos modificados o descartados (con justificación) 7
4. Arquitectura y decisiones técnicas 9
   4.1. Componentes 10
   4.2. Enrutado 10
   4.3. Decisiones técnicas 11
5. Funcionalidades implementadas 12
   5.1. Las funcionalidades, una a una: qué hace, cómo se usa y qué requisito cubre . 12
   5.2. Nada de lo que se entrega funciona con datos simulados 13
6. Seguridad del producto 13
   6.1. Secretos: el incidente de las claves 14
   6.2. Exposición de la caja: cómo estaba 14
   6.3. Endurecimiento aplicado al inicio de la práctica 14
   6.4. Autenticación y aislamiento: de cuatro fugas a cero 15
7. Pruebas y evidencias 19
   7.1. El plan 19
   7.2. Las pruebas del producto 21
   7.3. La plataforma y el despliegue 26
   7.4. El módulo de análisis con IA, paso a paso 28
8. Matriz de trazabilidad 31
9. Limitaciones y trabajo futuro 32
   9.1. El estado de sesión es volátil, por diseño 33
   9.2. El techo de gasto de la IA: aplicado, y lo que sigue sin estar acotado 33
   9.3. Lo que queda de la confusión de escala, una vez cerrada 33
   9.4. El manual de uso va por dos pestañas escritas y tres por escribir 33
   9.5. Dos cosas del Spider que se dejan dichas, no hechas 34
   9.6. El Repeater trabaja con una petición cada vez, y es una decisión 34
   9.7. El borrador del Repeater guarda las credenciales del objetivo en el navegador . 34
   9.8. Lo que falta para igualar al Repeater de referencia 35
   9.9. El frontend no pasa su propio analizador estático 35
   9.10. Restos del laboratorio retirado 35
   9.11. La herramienta que genera los esquemas no está versionada 36
   9.12. El vídeo de demostración no se entrega 36
10. Reparto del trabajo 36

11. Uso de herramientas de IA 38
11.1. Los límites de la herramienta, observados en primera persona 38
11.2. Los defectos que introdujo el asistente al arreglar otros 39
11.3. Qué aportó el asistente, tramo a tramo 39
11.4. Generado frente a verificado: la regla, y lo que cuesta sostenerla 40
12. Anexos 40
12.1. Manual de despliegue 40
12.2. Usuarios de prueba y acceso para la corrección 42
12.3. Glosario 42
12.4. Referencias 43

SECCIÓN 1

Resumen ejecutivo

HookSuite es una herramienta de auditoría de seguridad web que se usa desde el navegador, sin instalar nada en el equipo del auditor. Reúne en un solo panel el rastreo de un sitio, el reenvío manual de peticiones, la prueba masiva de valores sobre un parámetro, un juego de utilidades de codificación y un módulo de inteligencia artificial que clasifica los hallazgos.

Para quién. Para quien audita aplicaciones web y no quiere depender de una herramienta instalada en su equipo. Se abre una dirección, se entra con un usuario propio y se trabaja con datos aislados de los del resto de cuentas.

De dónde viene. La Práctica 1 dejó un prototipo que funcionaba, pero que no era defendible. La verificación en vivo del prototipo lo dejó claro: la API respondía desde internet sin pedir credenciales, la credencial documentada no abría el panel y cuatro claves de la API de Anthropic estaban dentro del historial de git. Esta práctica parte de ahí y lleva el prototipo a un estado instalable, desplegado y verificable. El recorrido completo está en el apartado 2.

En qué estado se entrega. De los veintiún requisitos numerados, trece quedan cumplidos, cuatro parciales, dos modificados, uno descartado y uno pendiente.

Los cuatro parciales, uno a uno. El navegador real con el que se ejecutan los ataques dirigidos (RF-10) está construido y conectado a la red interna, pero su prueba no se ha llegado a ejecutar. El historial del repositorio no refleja un reparto de tareas, sencillamente porque no lo hubo: la autoría de cada confirmación se sortea, y el apartado 10 lo explica. El análisis con inteligencia artificial tiene el recorrido verificado de punta a punta y el techo de gasto ejecutado, pero todavía no ha llegado a mostrar un hallazgo analizado, porque todas las auditorías que han terminado el recorrido han salido degradadas. Y la seguridad del propio producto queda parcial por las dependencias: acumulan avisos conocidos y en esta práctica no se actualizó ninguna, el apartado 6 da los números.

El requisito pendiente es el cifrado del tráfico, así que el producto se sirve hoy por HTTP. Lo demás funciona de principio a fin: alta y acceso por usuario con datos aislados entre cuentas, rastreo del objetivo, reenvío manual de peticiones, prueba masiva de valores sobre un parámetro y utilidades de codificación.

SECCIÓN 2

Punto de partida

HookSuite nació en la Práctica 1 como prototipo de una suite de auditoría web al estilo de Burp Suite, operada desde el navegador y asistida por inteligencia artificial. Quedó desplegado en un servidor de Hetzner con Docker Compose y, desde la entrega de mayo de 2026, nadie volvió a tocarlo. La Práctica 3 lo retoma tal como quedó.

Qué nos encontramos

La verificación en vivo del prototipo dio este cuadro.

- La herramienta seguía en pie desde mayo y sin cambios, con el servidor web y la API respondiendo.
- El núcleo funcionaba: rastreo, reenvío manual, prueba masiva de valores, utilidades de codificación, sesiones por token y canal de actualización en vivo.
- La API respondía en el puerto 8000, abierta a internet y sin ninguna autenticación, incluidas las rutas que lanzan tráfico contra terceros.
- En el historial de git había claves de API reales.
- El módulo de inteligencia artificial, que es lo que da sentido al producto, nunca llegó a operar más allá de unas pruebas iniciales. El panel estaba construido, pero inactivo.

Los dos hallazgos de seguridad reorientaron el trabajo de esta práctica y se tratan en el apartado 6.

La lección más cara de la Práctica 1

En las primeras pruebas del módulo de inteligencia artificial se cargaron cinco euros de saldo en la API y se agotaron en muy poco tiempo: el módulo ejecutaba una auditoría completa al arrancar y el servicio estaba declarado para reiniciarse solo, así que cada vez que terminaba volvía a empezar. Se corrigió en mayo haciendo que el módulo espere instrucciones. Figura en el punto de partida porque esta práctica hereda el problema a medias: el disparo automático quedó curado, pero el tamaño de una auditoría siguió sin acotarse. El techo de gasto que lo resuelve se explica en los apartados 7 y 9.

Numeración de requisitos

La Práctica 1 no numeraba los requisitos. El enunciado de la Práctica 3 obliga a numerarlos como RF-xx y RNF-xx y a usar esa numeración en toda la práctica. La lista completa está en el apartado 3 y está derivada del informe de la Práctica 1, de lo que el producto prometía y de lo que realmente hacía.

Clasificación inicial de requisitos

Con las cinco categorías que define el enunciado. Esta tabla es la foto del punto de partida y no se ha retocado después.

Estado	Qué significa	Requisitos
Cumplido	Implementado en la P1 y funciona. En la P3 se verifica con prueba y evidencia	RF-01, RF-03, RF-04, RF-05, RF-06, RF-07, RF-11 · RNF-01, RNF-02, RNF-03, RNF-05, RNF-08 **(12)**
Parcial	Existe algo, pero incompleto o con fallos. Hay que terminarlo y probarlo	RF-08 · RNF-04, RNF-07 **(3)**
Pendiente	Definido, pero sin implementar. Se hace desde cero	RF-09, RF-10, RF-12 · RNF-09 **(4)**
A revisar	El feedback o la experiencia indican que estaba mal planteado. Se reformula con justificación	RF-02 · RNF-06 **(2)**
Descartado	Se decide no implementarlo. Debe ser la excepción y justificarse	ninguno

Dos requisitos acabaron en un estado que esta clasificación no preveía: RF-04 pasó de cumplido a **modificado** y RF-09 de pendiente a **descartado**. Los dos cambios se justifican en el apartado 3.

Sobre el feedback de la Práctica 1

El enunciado pide reunir la calificación y los comentarios del equipo docente y construir esta práctica sobre ellos. El grupo no recibió ni calificación ni comentarios, así que ese material no existe y no se puede citar. Queda dicho en lugar de dejar el apartado a medias.

Las mejoras salen entonces de dos fuentes propias: el plan de mejoras que la propia Práctica 1 dejó escrito, del que se abordaron los cinco primeros puntos, y la revisión crítica del producto hecha ejecutándolo, que encontró los defectos que esta memoria documenta y corrige uno a uno. Y una precisión sobre la tabla: la categoría A revisar se define por «el feedback o la experiencia». Sin feedback, en esta práctica fue siempre la experiencia, y por eso RF-02 y RNF-06 se reformularon por lo que se vio al usar el producto.

SECCIÓN 3

Requisitos

Lista completa y numerada. La numeración se fijó en esta práctica, porque la Práctica 1 no la tenía, y se derivó de su informe. La columna de estado parte de la clasificación inicial del apartado 2 y recoge los cambios que esta práctica fue introduciendo.

Requisitos funcionales (RF)

ID	Requisito	Estado (de partida, puesto al día durante la P3)
RF-01	Dashboard web accesible desde cualquier navegador, sin instalar nada en el cliente	Cumplido
RF-02	El servidor ejecuta las peticiones HTTP en nombre del auditor (proxy del lado servidor)	**Modificado**, justificado abajo
RF-03	Spider: rastreo automático del objetivo (BFS, scope al dominio, extrae formularios), velocidad configurable	Cumplido
RF-04	Auditoría autenticada: sesión httpx persistente compartida por Spider/Repeater/Intruder que hereda las cookies del login	**Modificado**, justificado abajo
RF-05	Repeater: modificar y reenviar cualquier petición, con respuesta en 4 vistas (Raw/Pretty/Preview/Headers)	Cumplido
RF-06	Intruder: fuzzing de parámetros marcados con un asterisco, 4 tipos (SQLi, Blind SQLi, XSS, genérico), con resultados en tiempo real	Cumplido
RF-07	Utilidades: Encoder/Decoder (Base64/URL/HTML/JWT), Hash (MD5/SHA1/SHA256/SHA512), Regex Tester, Payload Generator	Cumplido
RF-08	**Análisis y clasificación de vulnerabilidades con IA (Claude) según**	Parcial

	OWASP que alimenta el panel Vulnerabilidades	
RF-09	Captura de tráfico HTTP en tiempo real (DevTools / CDP) que alimenta el panel Red con código de colores	**Descartado**, justificado abajo
RF-10	Ejecución de ataques con navegador real (Playwright)	**Parcial**, construido y en la red interna, sin prueba ejecutada (apartados 6 y 7)
RF-11	Soporte multi-usuario simultáneo mediante tokens de sesión UUID	Cumplido
RF-12	Registro y login individual con JWT + aislamiento de historial/auditorías por usuario	Cumplido (mejora, no venía de la P1)

> **Matiz sobre el enunciado de RF-03.** El requisito habla de «velocidad configurable». El control que existe no regula velocidad: fija un tope de páginas a visitar. Un valor menor no rastrea más rápido, llega menos lejos. El enunciado se conserva tal como se escribió en la P1, porque no se reescriben requisitos a posteriori, pero la interfaz sí se corrigió para decir lo que de verdad hace, y el control pasó a llamarse «hasta N páginas».

Requisitos no funcionales (RNF)

ID	Requisito	Estado (de partida, puesto al día durante la P3)
RNF-01	Accesible desde navegador sin instalar software en el cliente	Cumplido
RNF-02	Actualización en tiempo real vía WebSocket (pings cada 30 s, eventos {type,payload})	Cumplido
RNF-03	Desplegado en servidor Hetzner con contenedores Docker	Cumplido
RNF-04	Código en GitHub con historial de commits que refleje el trabajo del grupo	Parcial
RNF-05	Estabilidad ante carga: límite de concurrencia para no saturar producción	Cumplido
RNF-06	Operación degradada cuando la IA no responde o no supera el umbral de confianza	**Cumplido**, justificado abajo
RNF-07	**Seguridad del propio producto**: secretos fuera del repo, control de acceso, validación de entradas, dependencias sin CVEs	Parcial
RNF-08	Firewall dinámico por sesión (abre un puerto por sesión, lo cierra al logout o tras 4 h)	Cumplido en la P1 y endurecido después, pero **no reejercitado en esta práctica**
RNF-09	TLS/HTTPS en los endpoints públicos (Let's Encrypt)	Pendiente

Requisitos modificados o descartados (con justificación)

El enunciado pide que todo requisito modificado o descartado lleve su versión original, la nueva, el motivo y las consecuencias.

RF-02: proxy del lado servidor (modificado)

- **Versión original:** interceptación del tráfico del navegador mediante archivo de autoconfiguración de proxy (PAC) y WebSockets.

- **Versión nueva:** el servidor ejecuta las peticiones directamente con httpx, sin interponerse en el navegador del auditor.
- **Motivo.** En la P1 el modelo PAC quedó expuesto y se saturó con tráfico de bots. El pivote a httpx directo elimina esa superficie y simplifica el flujo.
- **Consecuencias.** El fichero de autoconfiguración se retiró del código, junto con su ruta en el backend y sus dos bloques en Nginx, después de comprobar que ya no lo usaba nadie. Con él desaparece la única ruta que tenía que quedar sin autenticar, porque un navegador no envía credenciales al pedirla, de modo que ahora toda la API exige token sin excepciones.

RF-04: auditoría autenticada con sesión compartida (modificado)

- **Versión original:** tal como está escrito el requisito, una sesión httpx persistente compartida por Spider, Repeater e Intruder, que hereda las cookies del login del objetivo.
- **Versión nueva:** el Repeater deja de usar esa sesión compartida y envía una petición independiente, con exactamente las cabeceras que se ven en pantalla. Spider, Proxy e Intruder siguen compartiéndola.
- **Motivo.** El requisito describe un mecanismo, pero lo que persigue es poder auditar autenticado, y ese mecanismo rompía en el Repeater algo que vale más, la reproducibilidad. El cliente compartido enviaba cookies que no estaban en la petición y se quedaba con las que devolvía el servidor, así que dos envíos idénticos daban resultados distintos y lo que se veía en pantalla no era lo que salía por el cable.
- **Consecuencias.** El requisito se sigue cumpliendo, porque la sesión se hereda igual pero por un canal explícito: las cabeceras que el producto muestra son las realmente enviadas, con la cookie del objetivo visible y editable. El cambio destapó además un defecto anterior que el cliente compartido tapaba, los formularios del Spider viajaban con cabeceras incompletas y todos los POST devolvían 419. Se corrigió el origen y el fallo desapareció.

RNF-06: operación degradada (a revisar en la P1, cumplido y demostrado en la P3)

- **Versión original:** enunciado como principio en la P1, sin demostrar.
- **Versión nueva:** la operación degradada se demuestra haciendo que un fallo de la IA sea distinguible de un análisis correcto. El clasificador devuelve tres estados donde antes había dos: vulnerable con confianza suficiente, analizado y limpio, y no analizado acompañado del motivo.
- **Motivo.** Cuando la IA no podía analizar, el clasificador devolvía lo mismo que cuando había analizado y no encontraba nada, de modo que el panel decía «sin vulnerabilidades» sin distinguir un resultado de una ignorancia. Y si la llamada a la API fallaba, nadie capturaba la excepción y se interrumpía la fase de análisis entera. Lo uno no es operación degradada sino silencio, y lo otro no es degradación sino caída.
- **Consecuencias.** Los tres estados existen en el código y una prueba automática los recorre en las cuatro vías de análisis, sin clave de API ni red y con coste cero. El estado degradado se ve en el panel, separado de los hallazgos y con su motivo, como muestra la figura del apartado 7.

RF-12: registro y login individual con JWT (mejora, no requisito P1)

- **Versión original:** no existía. No era requisito de la Práctica 1, venía del plan de mejoras que esa misma práctica dejó escrito.
- **Versión nueva:** registro y acceso individual con JWT, con los datos de cada cuenta aislados del resto.
- **Motivo.** Se aborda por su valor de seguridad, porque es lo que cierra la fuga de datos entre usuarios documentada en los apartados 6 y 7. Cuenta como mejora, que suma pero no compensa un requisito original sin cumplir.

- **Consecuencias.** La persona se da de alta ella misma y elige su propia contraseña, que nadie más conoce. Hace falta además un código de invitación que reparte el grupo, porque HookSuite lanza tráfico contra terceros y con altas anónimas cualquiera podría atacar desde nuestra infraestructura. El código es obligatorio por configuración: si faltara, el servicio no arranca, en lugar de arrancar con el registro abierto sin que nadie lo note.

RF-09: captura de tráfico en tiempo real con DevTools/CDP (descartado)

- **Versión original:** captura del tráfico HTTP en tiempo real mediante el protocolo de depuración de Chrome (CDP), volcada a un panel «Red» con código de colores por método y estado.
- **Versión nueva:** se descarta. La pestaña Red se retiró del producto, con su entrada de navegación, su ruta y su página, y la barra pasó de seis pestañas a cinco. Después se añadió la pestaña Manual, así que el producto que se entrega vuelve a tener seis: Spider, Repeater, Intruder, Utilidades, Vulnerabilidades y Manual.
- **Motivo.** Verificado contra el código antes de decidir: RF-09 no era una pantalla a medio pulir sino un vertical entero sin cerrar, con cuatro cosas rotas a la vez (el capturador busca el navegador en rutas de Windows que no existen en los contenedores, sus hallazgos de consola van a un extremo inexistente, su lanzador apunta a un puerto retirado y al fichero PAC jubilado con RF-02, y la pestaña era pasiva, sin forma de distinguir «no hay tráfico» de «el capturador no corre»). Cerrarlo exigía arreglar las cuatro y, sobre todo, probarlo, con el plazo de entrega encima.
- **Consecuencias.** El capturador permanece en el repositorio como punto de partida para más adelante. Los extremos de red del backend también se conservan, y no por inercia: los consume el módulo de IA y el de Playwright, así que retirarlos habría roto RF-08. RF-01, el acceso desde el navegador sin instalar nada, se mantiene cumplido. La pestaña Red tal como estaba quedó capturada a propósito antes del arreglo, porque el propio arreglo destruía la evidencia, y esa captura se conserva en docs/capturas/devtools/devtools_red_tiempo_real.png del repositorio.

SECCIÓN 4

Arquitectura y decisiones técnicas

HookSuite se despliega como un conjunto de contenedores Docker coordinados por docker-compose, todos en una red interna hooksuite-net, con Nginx como única puerta de entrada. La arquitectura se verificó fichero a fichero sobre docker-compose.yml y infra/nginx.conf. El producto vive en dos instalaciones iguales, una de desarrollo y otra pública, y la pública solo recibe despliegues: nada se prueba sobre la instalación que usan las personas, porque esta herramienta lanza tráfico contra sistemas de terceros.

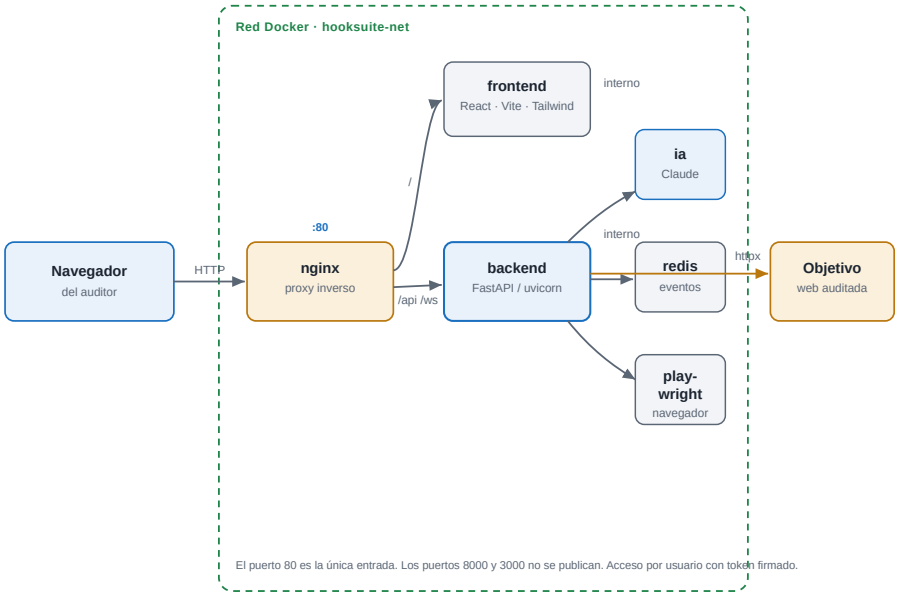


Figura 1: Arquitectura de contenedores de HookSuite. Nginx es la única entrada. El backend orquesta el rastreador, el banco de peticiones, la prueba masiva de valores, el módulo de IA y el navegador real, y sale al objetivo auditado. La autenticación no está en Nginx sino en el backend, que exige token en toda la API y en el canal en tiempo real (apartado 6).

Componentes

Componente	Tecnología	Rol	Puerto publicado
nginx	nginx:alpine	Proxy inverso y única puerta de entrada. Reparte entre frontend y backend	configurable, 80 por omisión
frontend	React, Vite y Tailwind (imagen propia)	Panel web servido como estático	ninguno
backend	FastAPI y uvicorn (imagen propia)	API de auditoría: proxy, rastreador, banco de peticiones, prueba masiva de valores, utilidades, vulnerabilidades y canal en tiempo real	ninguno
redis	redis:7-alpine	Bus de eventos y estado efímero entre servicios	ninguno
ia	Python con el SDK de Anthropic (imagen propia)	Clasificador de vulnerabilidades con Claude (RF-08)	ninguno
playwright	Python y Playwright (imagen propia)	Navegador real para ataques dirigidos (RF-10)	ninguno

Solo el proxy publica puerto. Los demás servicios se hablan por la red interna y no son alcanzables desde fuera.

Enrutado

Toda petición entra por Nginx, que reparte según la ruta pedida con la configuración de infra/nginx.conf.

- La raíz va al frontend, que sirve el panel de acceso del producto. Antes esa puerta iba protegida por una credencial única del servidor web, igual para todos. Ya no.
- Las rutas de la API y del canal en tiempo real van al backend, y ambas exigen token. Sin él responden 401.
- Las dos rutas del fichero de autoconfiguración de proxy se retiraron al jubilar ese modelo (apartado 3, RF-02), y también se retiró la ruta hacia el laboratorio vulnerable (apartado 6).

El reparto de responsabilidades cambió durante esta práctica, y es una decisión de arquitectura y no un detalle. Antes Nginx era quien autenticaba, con una credencial única que no distinguía personas, y el backend no pedía nada. Ahora Nginx solo reparte, y quien autentica y autoriza es el backend, que es el único que puede saber de quién es cada dato.

Decisiones técnicas

Las cuatro de más peso, cada una con lo que se decidió, por qué y qué cuesta.

El bus interno entre el backend y el módulo de IA (RF-08)

- **Qué se decidió.** El panel pide la auditoría por la ruta normal, con su credencial. El backend publica la orden en un bus de eventos interno y el módulo de IA la recoge de ahí. Cada mensaje lleva el espacio de datos de quien pidió la auditoría, de modo que los hallazgos aterrizan en su sesión.
- **Por qué.** El control de acceso dejó fuera al propio módulo, que preguntaba por HTTP sin credencial y recibía un rechazo. Las otras dos salidas eran peores: una credencial de servicio abre una excepción justo en la pieza cuyo valor es no tenerlas, y dejar la ruta sin control permitiría a cualquiera encolar auditorías desde nuestra infraestructura. El bus no necesita credencial porque la frontera la pone la red: no está publicado.
- **Qué cuesta.** Un control de acceso falla cerrando y un bus falla abriendo, así que un mensaje sin dueño se descarta. Hay pruebas de eso y de que dos auditores no ven lo del otro (apartado 7).

El Repeater se separa de la sesión compartida y de las redirecciones (RF-05)

- **Qué se decidió.** La función de envío que comparten las cuatro herramientas gana dos interruptores que solo el Repeater cambia: no seguir las redirecciones y no usar el cliente compartido.
- **Por qué.** Un Repeater que sigue las redirecciones nunca enseña el 302 ni su Location, que en auditoría es justo lo que interesa. Y el cliente compartido custodia las cookies del objetivo, de modo que dos envíos idénticos no daban el mismo resultado. Justificación completa en RF-04, apartado 3.
- **Qué cuesta.** Nada a las otras tres herramientas, porque la librería de red permite decidir las dos cosas petición a petición. Se eligió eso antes que construir un segundo cliente, que habría duplicado la configuración de red.

Del mismo cambio salió un arreglo que beneficia a las cuatro herramientas: las cabeceras que el producto muestra como enviadas se leen ahora de la petición que la propia respuesta conserva, no de un diccionario construido aparte, y la evidencia que se muestra es la que acaba copiada en un informe.

Del fichero de autoconfiguración de proxy a httpx (RF-02)

- **Qué se decidió.** El servidor ejecuta las peticiones directamente con httpx, en lugar de interceptar el tráfico del navegador del auditor.
- **Por qué.** Es la decisión de arquitectura de más peso heredada de la Práctica 1. Aquel modelo quedó expuesto y se saturó con tráfico de bots. El cambio reduce la superficie y simplifica el flujo.
- **Qué cuesta.** Se pierde la captura pasiva del tráfico del navegador, que es justo lo que retoman después DevTools (RF-09) y el navegador real (RF-10). Versión original, nueva y justificación en el apartado 3.

SECCIÓN 5

Funcionalidades implementadas

Cada funcionalidad con lo que hace, cómo se usa y qué requisitos cubre. El panel tiene seis pestañas más la pantalla de acceso, y el módulo de IA trabaja por detrás, sin pestaña propia.

Las funcionalidades, una a una: qué hace, cómo se usa y qué requisito cubre

Esta tabla cuenta lo que hay, pantalla por pantalla, tal como queda en la versión que se entrega. La columna de la derecha dice dónde se ve cada una, sin repetir aquí figuras que ya están en otros apartados.

Pantalla	Qué hace y cómo se usa	Requisitos	Dónde se ve
Spider	Rastrea el objetivo siguiendo enlaces y formularios y levanta su mapa. Se escribe la dirección, se lanza, y la tabla se va llenando. Se puede detener a medias, filtrar por método y mandar una petición al Repeater. Las cabeceras que muestra son las que se enviaron de verdad, no una plantilla	RF-03	Figura 3
Repeater	Banco de trabajo de una petición: se modifica y se reenvía exactamente lo que se ve en pantalla. Avisa si la petición lleva sesión heredada y ofrece quitarla. El botón «preparar» convierte la plantilla de un formulario en una petición que funciona. La respuesta se lee en Raw, Pretty, Preview, Headers y la página preparada	RF-04 · RF-05	Figura 4, Figura 5 y Figura 6
Intruder	Prueba muchos valores en un punto marcado de la petición y señala las respuestas que se salen del patrón. Se marca el campo, se elige diccionario, se lanza y se puede cancelar sin dejar el servidor trabajando	RF-06	Figura 7
Utilidades	Cuatro herramientas que no salen a la red: codificador (base64, URL, HTML y lectura de JWT), generador de huellas, probador de expresiones regulares y generador de payloads. Se pega el texto y se ve el resultado al lado	RF-07	Figura 8
Vulnerabilidades	Panel donde se dispara la auditoría con IA y donde aterriza su resultado. Lo que está visto funcionando es el camino degradado: lo que la IA no pudo analizar sale aparte y con su motivo. Nunca se ha visto un hallazgo analizado con su severidad, porque ninguna auditoría ha llegado a devolver uno. La pantalla queda sin pulir y sin terminar	RF-08 · RNF-06	Figura 11
Manual	Manual de uso dentro del propio producto, control por control y navegable por pastillas. No lo pide ningún requisito: salió del pulido, al comprobar que explicar una pantalla destapa sus defectos	no aplica (mejora)	Figura 13
Acceso (fuera de la barra)	Alta con código de invitación y entrada con token firmado por el servidor. Cada cuenta ve solo sus propios datos	RF-11 · RF-12	Figura 2, Figura 9 y Figura 10

Tres piezas trabajan sin pestaña propia, y quien busque su pantalla no la va a encontrar: el proxy del lado servidor (RF-02) es el que ejecuta las peticiones por debajo de Spider, Repeater e Intruder. El navegador real (RF-10) lo dirige el módulo de IA, no una persona. Y el

cortafuegos por sesión (RNF-08) es un agente del sistema. La séptima pestaña que hubo, Red (RF-09), se retiró al descartar ese requisito, con su captura tomada antes de retirarla y conservada en el repositorio.

Tres notas de honestidad sobre esta tabla.

- **La pestaña que se llama Spider vive en la ruta /proxy**: un resto del nombre anterior al pivote de RF-02, y se deja dicho porque cualquiera que mire las rutas lo va a notar.
- **El manual interno cubre dos de sus cinco secciones**: Spider y Repeater escritas, y las otras tres marcadas como pendientes dentro del propio producto, para que quien usa la herramienta vea que falta.
- **El pulido de este tramo llegó a dos pantallas, no a seis.** Spider y Repeater se repasaron enteros (quince y dieciséis defectos, apartado 7). Vulnerabilidades se empezó y no está terminada, y así se dice en su fila. E Intruder y Utilidades se entregan como estaban en la Práctica 1: el único cambio del Intruder es su cancelación corregida, y el de Utilidades es documental, porque la lectura de JWT ya existía y lo equivocado era la nota de la verificación inicial, que la daba por ausente. RF-07 queda cumplido sin salvedades.

Nada de lo que se entrega funciona con datos simulados

El enunciado pide que, si una integración externa no está disponible y hay que simularla, se indique de forma explícita. Aquí no hace falta simular ninguna, y conviene decirlo con los interruptores delante porque el código sí trae los mecanismos para hacerlo.

- **La única integración externa del producto es la API del modelo**, y no se simula. Cuando falta la clave o la llamada falla, el producto no inventa un resultado: devuelve el estado degradado con su motivo (RNF-06, apartado 7). Es exactamente lo contrario de simular.
- **El navegador real se usa de verdad.** El interruptor MOCK_PLAYWRIGHT está en false en los dos servicios del fichero de composición y en el de ejemplo, así que la automatización va contra un navegador y no contra una respuesta escrita a mano.
- **El panel no sirve datos de ejemplo.** El interruptor USE_MOCKS está fijado a false en el código del frontend y ninguna variable de entorno lo enciende: haría falta editar el código y reconstruir.
- **Lo que sí hay que declarar, porque es verdad y es un defecto menor:** el módulo de datos de ejemplo sigue viajando dentro del paquete del frontend aunque nadie lo muestre, y menciona un laboratorio que ya salió del producto. No se ve en pantalla, pero está en lo entregado. Se retira después de la entrega.
- **El cliente simulado del modelo existe solo en las pruebas** (ia/tests/test_tres_estados.py), para poder demostrar RNF-06 sin gastar saldo, y se declara así en el apartado 7. No forma parte del producto.

SECCIÓN 6

Seguridad del producto

Modelo de amenazas, secretos, autenticación, validación de entradas, dependencias y datos personales. Todo lo que sigue está verificado en esta práctica:

Secretos: el incidente de las claves

En el historial de git había claves de API de Anthropic. Comprobado contra la API: las cuatro están muertas (la API responde 401). Aun así el enunciado obliga a limpiarlas del historial, no basta con borrarlas en un commit nuevo. Se reescribió el historial con git-filter-repo (claves redactadas por patrón, .env rastreados y basura fuera, conservando código y las 7 ramas) y se ejecutó en GitHub con force-push y copia de seguridad previa. Verificado sobre un clon nuevo: 0 claves en todo el historial. Salvedad declarada: GitHub conserva un tiempo los refs internos de los pull requests antiguos y los purga por su cuenta. Como las claves están muertas, no supone riesgo.

Exposición de la caja: cómo estaba

Estado de partida, verificado en vivo sobre el prototipo. Las cinco capturas que lo documentan están en el repositorio y el apartado 7 describe qué muestra cada una:

- La API :8000 no tenía autenticación: lo declaraba su propia openapi.json (40 operaciones sin seguridad) y /health respondía sin credenciales.
- El Basic Auth del :80 se evitaba sirviendo el frontend directamente por :3000.
- El servidor no tenía cortafuegos (política de aceptación por defecto) ni protección contra fuerza bruta, y acumulaba unos 127.000 intentos de acceso en 7 días.
- DVWA no estaba expuesto a internet (solo red interna), lo que corrige un supuesto anterior.

Endurecimiento aplicado al inicio de la práctica

Cierre de la exposición

La API y el frontend dejaron de publicar puerto propio: la única entrada es el proxy. No bastaba con retirarlos del docker-compose.yml, porque el frontend llevaba la dirección del servidor incrustada en tiempo de compilación y hablaba directamente con la API. Hubo que reconstruirlo para que use el mismo origen y que sus llamadas (incluido el WebSocket) pasen por el proxy. Comprobado desde fuera tras el despliegue: los puertos directos rechazan la conexión, el proxy responde, y el paquete del frontend ya no contiene ninguna referencia a la dirección antigua.

Orígenes permitidos (CORS)

El backend aceptaba cualquier origen con credenciales, combinación además inválida por especificación: en la práctica, el servidor reflejaba el origen de quien preguntara, de modo que cualquier web podía hacerle peticiones autenticadas. Se sustituyó por una lista cerrada configurable por entorno. Comprobado: un origen arbitrario ya no recibe permiso. Uno legítimo sí.

El laboratorio vulnerable: fuera del producto

Primero se le retiró del proxy la ruta que lo publicaba, de modo que quedó accesible solo desde la red interna. Al cerrar el desarrollo se retiró del producto entero: no está en el despliegue ni en la caja, ni el servicio, ni el contenedor, ni la imagen. Las pruebas de este proyecto se hacen contra una web propia del equipo, autorizada para ello, y mantener en el despliegue un sistema deliberadamente vulnerable que ya nadie usaba no tenía defensa. El único punto del código que todavía caía a él cuando llegaba una orden sin objetivo ahora descarta esa orden y lo dice, en lugar de auditar un host que ya no existe. Lo que no se tocó, y se declara: el módulo de navegador conserva una función de acceso a ese laboratorio y los datos de ejemplo del frontend lo siguen nombrando. Es código que no se ejecuta en el producto entregado, y reescribirlo con el desarrollo ya cerrado añadía riesgo sin ganar nada. Queda anotado en el apartado 9.

Endurecimiento del servidor

- **Cortafuegos** activo, con política de denegación por defecto y únicamente el acceso remoto y el proxy permitidos.
- **Bloqueo automático de fuerza bruta** contra el acceso remoto: en su primer minuto ya registraba 148 intentos fallidos y había bloqueado dos direcciones.
- **Acceso remoto solo por clave**: se desactivó la autenticación por contraseña. La prueba más limpia es el propio mensaje del servidor, que pasó de ofrecer «clave o contraseña» a admitir solo clave.
- **Limpieza de reglas abandonadas**: siete reglas heredadas de la P1, una con la dirección pública de un integrante, se retiraron tras comprobar que ningún servicio escuchaba en esos puertos.
- **Agente de firewall duplicado**: un segundo proceso del agente, lanzado a mano en mayo y fuera del gestor de servicios, con permisos de administración del cortafuegos, se terminó. Queda uno solo, gestionado por el sistema.
- **Sistema operativo**: se estrenó el núcleo ya instalado pero sin aplicar desde mayo, corrigiendo antes una condición de carrera en el arranque (apartado 7).

Dos matices honestos, para no atribuirnos de más

El acceso de administrador por contraseña ya estaba bloqueado por la configuración por defecto del sistema: los 127.000 intentos registrados nunca tuvieron posibilidad, y lo que se cerró fue la autenticación por contraseña a nivel general, para no depender de un valor por defecto que una actualización podría cambiar. Y el cortafuegos del sistema no filtra los puertos que publica Docker, cuyas reglas se evalúan antes: quien cerró de verdad la API y el frontend fue el despliegue, al dejar de publicarlos.

Autenticación y aislamiento: de cuatro fugas a cero

Hasta que se rehízo el acceso, la API y el WebSocket no tenían autenticación por detrás del proxy: el único control era el Basic Auth de la entrada, que además no distinguía personas. Eso no era una carencia, sino cuatro fallos distintos, los cuatro medidos en el entorno local de pruebas antes de tocar una línea de código. La tabla enfrenta cada uno con lo que lo cierra. La evidencia de los dos lados está en el apartado 7.

Cómo estaba (medido)	Cómo queda (acceso por usuario)
La API entera respondía sin credencial. GET /health devolvía 401 por el Basic Auth de la entrada, pero GET /api/session/new devolvía 200 y entregaba un token: el proxy protegía unas rutas y dejaba /api abierta.	Un guardián aplicado **por router**, no ruta por ruta, para que una ruta nueva nazca protegida. Toda /api y /ws/ exigen token. El Basic Auth de la entrada se retiró.
El token de sesión no era una credencial, era un nombre. El backend creaba sesión para cualquier cadena inventada: bastaba escribir un token cualquiera en la URL.	El token lo **firma el servidor** y el espacio de datos se deduce del token, nunca de la URL ni del cuerpo. La ruta que repartía identificadores sin valor de credencial se retiró.
La cookie de sesión de la web auditada se compartía entre todos. Vivía en una tabla global sin dueño: un segundo usuario leía la cookie capturada por el primero, además sin token.	Las cookies viven dentro de la sesión de cada usuario y la tabla global desapareció. Verificado: el segundo usuario ya no la obtiene.
Los hallazgos se difundían a todos los paneles. Una vulnerabilidad publicada sin token aparecía en la sesión de dos auditores distintos: una fuga de los datos que otro estaba auditando.	Los dos endpoints que admitían peticiones sin token escriben ahora solo en el espacio de quien llama. Verificado: el segundo usuario no ve el hallazgo del primero.

Un quinto fallo, que introdujo la propia corrección

El guardián comprobaba que el espacio de datos nombrado en la ruta fuera el tuyo, pero cinco rutas reciben ese nombre en el cuerpo de la petición, y ahí no miraba nadie: un usuario

autenticado podía escribir en el espacio de otro con solo ponerlo en el cuerpo. La peor de las cinco fue el Repeater, que mantiene un cliente HTTP persistente por usuario con las cookies del objetivo auditado: nombrar a otro en el cuerpo reutilizaba su sesión ya autenticada contra la web auditada. Se descartó la corrección evidente, validar ese campo contra el token, porque deja el dato en manos del cliente: las rutas toman el espacio del token y descartan lo que venga en el cuerpo, y lo que no se lee no se puede falsear. Comprobado como ataque real (apartado 7).

Desplegado y verificado en producción

Hasta entonces todo estaba verificado solo en el entorno local, y después se comprobó en vivo y en la propia instalación pública que las fugas no eran teóricas: la API respondía sin credencial a cualquiera. El despliegue se hizo en cuatro pasos, con punto de retorno previo: se etiquetaron las imágenes en servicio y se copió la configuración, se mergeó el cambio a la rama principal, se trajo el código a la caja con git reset –hard y se generaron los secretos, y se reconstruyeron los contenedores. La verificación inmediata, con el sistema ya en marcha:

Qué se midió	Resultado
Entrada del panel (/)	200, carga **sin pedir Basic Auth**: la contraseña compartida está retirada
API (/api/auth/yo)	401, **exige token**. La fuga nº 1 de la columna izquierda, cerrada
Reinicios del backend	0, lo que prueba que los secretos llegaron: el código se niega a arrancar sin ellos
Contenedores recreados	backend, frontend y nginx. Los otros cuatro, intactos a propósito

El dato de los cero reinicios no es decorativo: el acceso por usuario se diseñó para no arrancar si faltan el secreto de firma o el código de invitación, en lugar de arrancar sin autenticación pareciendo correcto. Que el contenedor esté sirviendo sin un solo reinicio es la prueba de que esa comprobación se superó.

El acceso, capturado en la instalación pública el mismo día del despliegue (la entrada carga ya sin el cuadro de Basic Auth que antes pedía una contraseña compartida por todo el grupo):

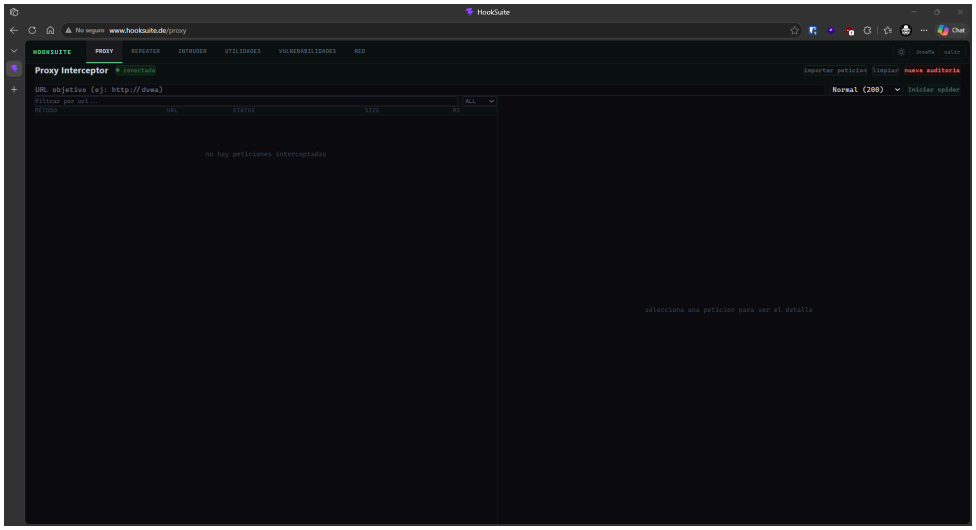


Figura 2: RF-12 y RNF-07 · sesión iniciada en producción, con el alta hecha mediante el código de invitación sobre una instalación que arrancó deliberadamente con cero usuarios: el usuario figura en la cabecera y el interceptor aparece como conectado, lo que prueba que el WebSocket viaja también autenticado y no solo la API.

Dos efectos del pulido del Repeater sobre la seguridad del producto

- **Cerrado: el último residuo del modelo anterior al login.** La petición de envío del Repeater llevaba en su cuerpo el identificador del espacio de datos, y el esquema del servidor seguía declarándolo. No se leía, el espacio sale siempre del token firmado, nunca del cuerpo, pero sugería que el cliente puede elegir dónde escribe, que es exactamente el hueco de autori-

zación que cerró el acceso por usuario: con el nombre de otro en el cuerpo se reutilizaban sus cookies del login auditado. Se retiró del frontend y, esa misma noche, del esquema del servidor. Un campo muerto que insinúa una capacidad que no existe es deuda de seguridad, no ruido.

- **Nuevo, y es una contrapartida asumida: el panel muestra ahora la cookie de sesión del objetivo.** Al corregir que las cabeceras mostradas sean las realmente enviadas, la Cookie del sitio auditado pasa a estar a la vista, como en la herramienta de referencia. Es lo correcto, la evidencia tiene que ser la petición que salió, y tiene un coste que conviene declarar: cualquier captura de ese panel publica credenciales de un tercero, así que esas capturas van censuradas antes de entrar en esta memoria. Lo que esto arrastra en el navegador del auditor se trata como limitación abierta en el apartado 9.

Lo que sigue sin cubrir

- **Sin TLS** (RNF-09): el acceso viaja en claro, así que el token de sesión es interceptable por quien esté en el camino. Pendiente de configurar en el proxy.
- **Credencial de servicio para los módulos internos.** Lo que este punto afirmaba se quedó atrás dos veces y se corrigió, y hoy son dos situaciones distintas. El clasificador de IA ya no necesita credencial: se le movió al bus interno precisamente para eso (apartado 4) y que RF-08 funcione de punta a punta es la prueba, porque si su camino de vuelta diera 401 el panel estaría vacío. El agente de navegador sí sigue sin credencial, y ese es el hueco real: sondea por HTTP las rutas protegidas sin cabecera de autorización y recibe 401. Queda como trabajo pendiente de RF-10, y la salida no es repartir una credencial más, es pasarlo al mismo bus que el clasificador.

Modelo de amenazas (STRIDE)

El modelo se aplica a los seis componentes del apartado 4 y a la frontera que movió el acceso por usuario: hasta entonces autenticaba el proxy con una contraseña compartida. Hoy autentica el backend, por persona.

La premisa de partida: HookSuite es una herramienta ofensiva, y su función es enviar tráfico arbitrario a un objetivo que elige quien la usa. Por eso el control no puede ser «no permitir destinos arbitrarios», eso sería desactivar el producto, sino **que solo entren personas autenticadas y que cada una responda de lo suyo**. Lo que en otra aplicación sería una vulnerabilidad (hacer peticiones a donde diga el usuario) aquí es el requisito RF-02.

Amenaza	Dónde aplica en HookSuite	Qué lo contiene hoy	Qué queda abierto
S · Suplantación	Entrar como otro auditor. Hablar con la API sin ser nadie	Entrada única por el proxy. /api/ y /ws/ exigen token firmado, con el algoritmo fijado en el servidor. Alta con código de invitación. Contraseñas en hash	**Sin TLS, el token es interceptable** (RNF-09). El agente de navegador sondea sin credencial propia (arriba)
T · Manipulación	Tocar datos de otra cuenta. Manipular al agente de cortafuegos	El espacio de datos **sale siempre del token**, nunca de la ruta ni del cuerpo. Modelos declarados que rechazan la petición mal formada antes de la lógica. El canal del agente tiene permisos acotados y valida lo que recibe	En claro y sin TLS, manipulable en tránsito
R · Repudio	Saber quién lanzó qué contra un objetivo	Cada petición, cookie y hallazgo cuelga de la cuenta que lo generó	**No hay registro de auditoría propio**: los rastros son los del contenedor, no se conservan ni se firman. Se declara, no se resuelve

I · Divulgación	Ver auditorías ajenas. Filtrar cookies del objetivo	Cuatro fugas cerradas y una quinta introducida y corregida. Las cookies del objetivo cuelgan de su dueño. Claves fuera del historial. Orígenes restringidos	Sin TLS todo viaja en claro. El borrador del Repeater guarda la cookie del objetivo en el navegador (apartado 9)
D · Denegación de servicio	Agotar el servidor, el saldo de la API o martillear al objetivo	Límite de concurrencia (RNF-05). **Techo de llamadas** a la IA con degradación visible. El recolector de sesiones ahora corre. En el host, cortafuegos y bloqueo de fuerza bruta	La vía de huella digital no compara contra el techo. El Spider no tiene límite de ritmo propio
E · Elevación de privilegio	Que un usuario del producto mande sobre el cortafuegos del host	El canal del agente, con permisos acotados y validación. Retirado un segundo agente lanzado a mano en mayo. Acceso remoto solo por clave	El agente de cortafuegos es, **por diseño**, un componente con privilegio sobre las reglas del host: esa superficie no desaparece

Validación de entradas

- **Modelos declarados en el borde, no comprobaciones dispersas:** el backend declara su entrada con modelos en seis ficheros, así que una petición mal formada se rechaza antes de llegar a la lógica.
- **Todas las rutas protegidas menos la puerta:** los nueve grupos de rutas de main.py llevan la dependencia de protección salvo el de alta y entrada, que es por donde se consigue el token.
- **El identificador del espacio de datos no se acepta del cliente:** las rutas lo toman del token y punto, lo que cierra la familia entera del quinto fallo.
- **Canonicalización en el Spider:** decodificar, normalizar y deduplicar. Sin eso, el rastreo generaba direcciones infinitas y martilleaba al objetivo (la trampa de araña).
- **En el módulo de IA, la validación es doble:** el esquema de respuesta endurecido tras los tres rechazos de la API, y el techo de llamadas cortando por arriba.
- **Lo que no se valida, y es deliberado:** la dirección del objetivo. Es la función del producto. El control está en el acceso, no en el destino.

Dependencias

Medido sobre el código que se entrega con npm audit sobre el fichero de bloqueo y pip-audit sobre los requisitos declarados, sin redondear hacia abajo.

- **Frontend:** 241 dependencias resueltas y 12 avisos, de los que ocho están en el árbol de producción, siete altos y uno bajo. Las directas son el cliente HTTP del panel, el enrutador y la herramienta de construcción.
- **Backend:** 78 avisos en 12 paquetes. El que más importa es la librería que firma los tokens de sesión, con trece en la versión fijada, por ser la pieza central del acceso por usuario. El proxy de la P1 aporta tres propios y arrastra cinco paquetes más, y no es código muerto: la ruta de alta sigue lanzándolo como subproceso aunque RF-02 se implemente hoy con el cliente HTTP nuevo.
- **Fijado desigual:** el backend fija sus doce versiones y el módulo de IA solo una de cuatro, así que su construcción no es reproducible hoy. El frontend declara rangos y el fichero de bloqueo fija el árbol.
- **Lo que no se ha hecho:** no se ha actualizado ninguna dependencia. La entrega se hace sobre esta etiqueta y subir la librería de tokens o el cliente HTTP cambiaría el código que se entrega. Queda declarado también en el apartado 9.
- **Honestidad de método:** esos 78 incluyen transitivas y no se ha medido cuáles son alcanzables desde el camino de ejecución, de modo que son avisos que hay que mirar y no

agujeros explotables. Los paquetes se nombran por su papel y no por nombre y versión porque el repositorio es público y el producto está desplegado sin TLS.

Tratamiento de datos personales

- **Qué guarda el producto, y de quién.** De cada auditor: su cuenta, nombre y contraseña en hash, en un volumen propio, y colgando de ella su historial, las cookies que captura del objetivo y sus hallazgos. Desde el cambio de acceso nada de eso se comparte entre cuentas.
- **De terceros, nada por iniciativa propia.** Lo que entra es lo que el auditor captura del objetivo que decide auditar, y las pruebas de esta práctica van contra una web propia del equipo y autorizada.
- **La cookie del objetivo es el dato más sensible que pasa por aquí**, porque equivale a una sesión abierta en la web auditada. Vive en el espacio de su dueño y el editor avisa cuando una petición la lleva, pero el borrador del Repeater la escribe en el almacenamiento del navegador: declarado en el apartado 9 y sin tocar el producto con el desarrollo ya cerrado.
- **Rastros a retirar antes de la entrega:** los usuarios de prueba del entorno local. La regla heredada de la P1 con la dirección pública de un integrante ya se retiró.
- **Conservación y borrado:** no hay caducidad ni borrado automático de cuentas ni de historial. Se declara como lo que es: una decisión no tomada, no una medida.
- **Y el que lo atraviesa todo:** sin TLS, todo lo anterior viaja en claro (RNF-09).

SECCIÓN 7

Pruebas y evidencias

Plan de pruebas, casos ejecutados, resultados y las pruebas que fallaron con su explicación. La matriz del apartado 8 relaciona cada caso con su requisito y su evidencia.

Dónde se tomó cada evidencia, porque cambia lo que vale. Salvo cuando se dice lo contrario, toda verificación se ejecutó primero en el entorno local de pruebas, una réplica del despliegue con los mismos contenedores en una máquina del equipo, y se repitió contra la instalación pública tras cada despliegue. Una prueba en el entorno local dice que el arreglo funciona. Solo la de producción dice que está puesto, y más abajo se cuenta un despliegue que habría pasado por correcto sin aplicar nada. Las salidas de terminal se guardan como texto en el repositorio y no como foto: son comparables entre versiones y ninguna captura publica por error un secreto.

El plan

El plan de pruebas: los dieciocho casos que cita la matriz

El plan original del equipo tenía doce casos, de modo que seis de los códigos que la matriz cita (PR-03, PR-04, PR-10, PR-12, PR-15 y PR-20) no tenían caso escrito: se redactan aquí por primera vez. Y los doce que existían llevaban los resultados medidos sobre el prototipo, de modo que varios eran hoy literalmente falsos (PR-01 figuraba como fallo por un control de acceso que ya no existe, PR-16 decía que toda la API respondía sin credenciales, PR-19 daba un error de servidor ya corregido). Esos resultados tampoco son basura: son la foto del punto de partida. Por eso la tabla lleva dos resultados, el del prototipo y el de hoy sobre el código que se entrega, y la celda de hoy dice además de dónde sale ese resultado: una reejecución,

una figura de esta memoria, o nada, y entonces se dice. Los códigos son los del plan de la práctica anterior y se conservan tal cual para que las remisiones de la matriz sigan valiendo, de ahí que la numeración llegue a veinte con dieciocho casos.

Cómo leer la tabla. De los dieciocho casos, seis se han reejecutado contra producción, seis los demuestra una figura de esta memoria, cinco no se han reejecutado y se dice por qué, y uno, PR-11, tiene el procedimiento caducado porque cita una ruta que el cambio de acceso retiró. En las filas reejecutadas, un 200 no se ha dado por bueno sin mirar el tipo de contenido, porque el servidor devuelve la portada del panel ante cualquier ruta desconocida. Las salidas completas están en docs/evidencias/pruebas-reejecutadas-13oct.md.

ID	Req.	Qué prueba	Prototipo	Código que se entrega
PR-01	RF-01	Abrir la URL pública en un navegador limpio	401 Basic Auth: la credencial documentada no entra	**200**, sirve el panel · reejecutada en producción
PR-03	RF-03	Rastrear un objetivo y revisar el panel de detalle	no ejecutada	**pasa** · Figura 3: rótulos legibles y cabeceras reales
PR-04	RF-04	Enviar dos veces la misma petición y comparar	**sin caso escrito**	**pasa** · Figura 5 y Figura 6: mismo estado, tamaño y cuerpo. Distinto tiempo
PR-05	RF-05	Modificar y reenviar una petición. Ver la respuesta	envío correcto, historial vacío	**pasa** · Figura 4. El historial se retiró por diseño (apdo. 9)
PR-06	RF-06	Marcar un campo y lanzar el Intruder	diccionarios servidos. Ejecución sin probar	**no reejecutada**: exige tráfico de ataque real y su procedimiento nombraba el laboratorio retirado
PR-07	RF-07	Hash, codificadores y regex con casos conocidos	hash y codificadores correctos. JWT no soportado	**no reejecutada**: exige credencial de usuario
PR-08	RF-08	Publicar un hallazgo y recuperarlo por el panel	backend correcto. Faltaba el productor	**pasa** · recorrido verificado y Figura 11 (techo a 0 euros)
PR-10	RF-10	Ejercitar el módulo de navegador	**sin caso escrito**	**no reejecutada**: el módulo ni siquiera estaba en la red interna hasta el último despliegue
PR-11	RF-11	Que cada usuario tenga su espacio de datos	tokens correctos por /api/session/new	**procedimiento caducado**: esa ruta se retiró con el acceso por usuario. Hoy RF-11 se prueba con PR-12
PR-12	RF-12	Que un usuario no vea los datos de otro	**sin caso escrito**	**pasa** · Figura 9 y Figura 10, verificado 11/11 en el entorno local
PR-13	RNF-03	Servicio en pie y API publicada en la caja	/health correcto. Esquema de la API **abierto**	**pasa y mejora**: /health 200 y el esquema **ya no se publica** · reejecutada
PR-14	RNF-07	Sin claves en lo que se publica	por ejecutar	**pasa** · cero ficheros seguidos por git con clave, con control positivo · reejecutada
PR-15	RNF-08	Cortafuegos por sesión	**sin caso escrito**	**no reejecutada**: el agente está operativo y endurecido (apdos. 6-7), pero no se ha vuelto a ejercitar
PR-16	RNF-07	Llamar a la API sin ninguna credencial	**toda la API respondía**	**pasa**: 401, y los puertos 8000 y 3000 rechazan conexión desde fuera · reejecutada
PR-17	RNF-02	Conexión en tiempo real con el servidor	enlace correcto, sin autenticar	**pasa** · el indicador «conectado» de Figura 2 lo prueba ya autenticado
PR-18	RNF-09	Responder por TLS	sin HTTPS	**sigue sin TLS** · reejecutada. RNF-09 es el único requisito declarado pendiente
PR-19	RF-02	Estado del proxy del lado servidor	**500**, error de servidor	**pasa**: 401, el extremo responde y exige credencial · reejecutada
PR-20	RNF-05	Límite de peticiones simultáneas	**sin caso escrito**	**no reejecutada**: el límite está implementado y citado en el apdo. 7, pero sin prueba propia

Las pruebas del producto

Pantalla por pantalla y requisito por requisito, con la evidencia capturada antes de cada arreglo.

Evidencia de exposición (RNF-07 / PR-16)

La prueba PR-16, llamar a la API sin credenciales, y la verificación del frontend confirman que, antes de endurecer, el producto quedaba accesible sin autenticación. Las capturas se obtuvieron antes de cualquier arreglo, porque la evidencia de un fallo se captura antes de corregirlo: el propio arreglo la destruye. Lo que documentan, pieza a pieza: GET /health de la API respondía 200 sin credenciales ni cabecera WWW-Authenticate. La documentación interactiva (/docs) quedaba abierta, exponiendo las 40 operaciones, incluidas las que lanzan tráfico contra terceros. El frontend servido por el puerto 3000 cargaba sin pedir credencial, evitando por completo el control de la entrada principal, que sí pedía credencial: el control existía, pero se podía rodear por el puerto directo. Y la credencial documentada en el informe de la P1 era rechazada con 401, ya no abría nada. Las cinco capturas, una por cada punto de los anteriores, están en docs/capturas/exposicion/ del repositorio, con el prefijo RNF07.

Los doce defectos del prototipo, corregidos y verificados

Todos se reprodujeron antes de corregirlos y se verificaron después en producción.

Qué fallaba	Causa	Requisito
El Spider entraba en bucle y martilleaba el objetivo	No canonicalizaba las direcciones: el mismo recurso generaba variantes infinitas	RF-03
El Spider podía dirigirse a servicios internos del propio servidor	Faltaba filtrar destinos no legítimos	RNF-07
El botón de detener el Spider no existía en la interfaz	El servidor aceptaba la orden, pero la pantalla no ofrecía forma de darla	RF-03
Detener el Spider no detenía nada	El bucle de rastreo no consultaba la señal de parada	RF-03
Un mismo formulario aparecía decenas de veces	No se agrupaban: se emitía uno por página visitada	RF-03
Cancelar el Intruder devolvía un error del servidor	La operación invocaba un método inexistente	RF-06
Dos operaciones del proxy devolvían error del servidor	Un error de sintaxis construía un tipo de dato equivocado	RF-02
Importar un comando curl rompía las direcciones seguras	Una expresión regular mal escapada recortaba el esquema	RF-05
Los avisos de error no decían qué había fallado	Se mostraba el texto crudo de la librería de red	RF-05
Dos pantallas no recibían avisos en tiempo real	El servidor guardaba una sola conexión por sesión y la segunda desplazaba a la primera	RNF-04
El agente del cortafuegos era manipulable por cualquiera	Su canal se creaba con permisos para todo el sistema y no validaba lo recibido	RNF-08
El estado de sesión podía crecer sin límite	El recolector existía pero nunca se ejecutaba, y cualquier identificador nuevo creaba una sesión	RNF-07

Dos de los doce quedaron además fotografiados antes y después de corregir, sobre el producto desplegado y contra una web propia del equipo: el panel que arrastraba 52 entradas de formulario idénticas por una sola petición queda en una, y el botón de detener que no existía aparece junto al de ejecución. Las cuatro capturas, antes y después de cada uno de los dos defectos, están en docs/capturas/fase1/ del repositorio, con el prefijo RF-03.

> Nota de tratamiento de las capturas. Las dos del estado anterior mostraban un identificador de sesión de la web de pruebas en la franja superior. Se publican con esa franja tapada, y los originales no salen del almacén interno. Las del estado posterior no lo contienen porque la sesión no estaba autenticada.

El pulido del Spider (RF-03): quince defectos, y catorce los encontró el uso

En este tramo no se añadió funcionalidad: se pulió lo que ya existía, pestaña por pestaña. De esa pasada salen quince defectos corregidos en el Spider, y lo que más dice de ellos es cómo aparecieron: catorce usando la herramienta y uno leyendo el código.

Nº	Qué pasaba	Cómo apareció
1	Pulsar «Detener» no paraba hasta terminar todos los formularios de la página en curso	usando
3	El indicador de sesión decía «autenticado» sin que nadie se hubiera autenticado	usando
7	La pestaña se llamaba «Proxy» y no es un proxy, sino un rastreador	usando
11	El objetivo fijado se perdía al refrescar la página, y permitía lanzar otro encima	usando
13	Las primeras filas capturadas no se alcanzaban con el scroll	usando
15	Los filtros escondían los formularios. Filtrar por POST vaciaba la tabla entera	leyendo el código

Son seis de los quince, los que mejor enseñan el patrón. Los quince completos, cada uno con su entrada del diario, están en docs/informe/diario.md del repositorio.

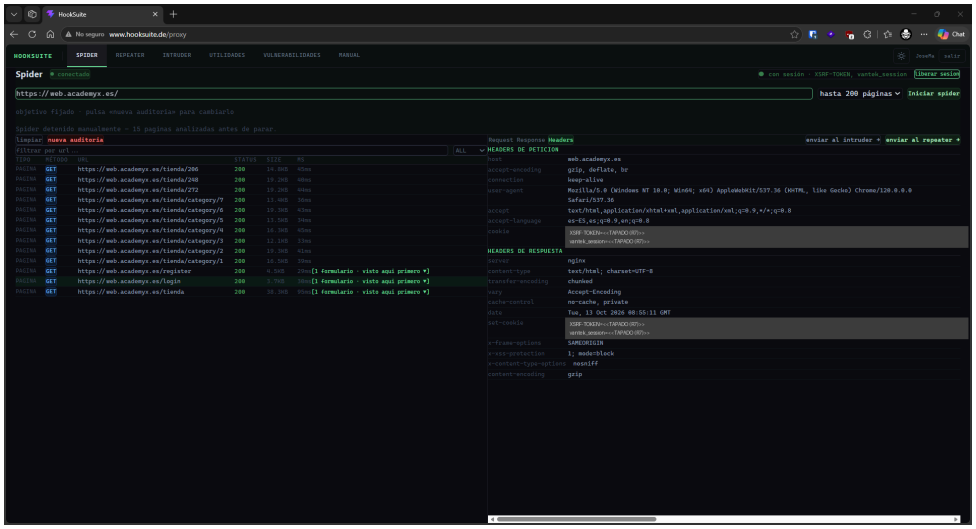


Figura 3: RF-03 · el panel de detalle después del pulido, con los dos defectos de fidelidad corregidos: los rótulos de sección se distinguen del contenido (antes compartían el mismo gris, y eso llevó a leer las cabeceras de respuesta como si fueran de petición) y las cabeceras que se muestran son las realmente enviadas, con la Cookie de sesión incluida. Los cuatro valores de sesión van tapados.

El patrón que se repite. En once de los quince, el backend ya sabía la respuesta y no llegaba a la pantalla: no eran fallos de cálculo sino de traslado, funciones correctas que la interfaz no dejaba ver, y las pruebas que miran el backend no detectan nada de esto. Dos de los quince los introdujimos nosotros al arreglar otros, y se detallan en el apartado 11.

El pulido del Repeater (RF-05): dieciséis defectos, y una función retirada el mismo día

El tramo siguiente repitió el método sobre la segunda pestaña: no se añadió alcance, se pulió lo que había. El resultado se parece al del Spider en el número y se diferencia en una cosa: una función se añadió y se retiró el mismo día.

Nº	Qué pasaba	Cómo apareció
1	Un fallo de red se mostraba como una respuesta más, con el código HTTP 0 y medidas de «0 ms» y «0,0 KB»	leyendo el código
5	El Spider guardaba y enseñaba como enviadas unas cabeceras **fabricadas aparte**: otro User-Agent, un Host que nadie puso y ninguna Cookie	tirando de un hilo
8	Las respuestas se cruzaban: se veía una dirección con el cuerpo de otra petición, y no era reproducible a voluntad	usando
10	El Repeater seguía las redirecciones siempre, así que **nunca** se veía el 302 ni su Location	comparando con la herramienta de referencia

11	Dos envíos idénticos daban resultados distintos: el cliente compartido ponía cookies que no estaban en la petición	usando
12	Los formularios descubiertos por el Spider viajaban sin sesión, y todos los POST devolvían 419	usando

Son seis de los dieciséis, los que mejor enseñan el patrón. Los dieciséis completos, cada uno con su entrada del diario, están en docs/informe/diario.md del repositorio.

La función que se añadió y se retiró el mismo día. El Repeater ganó un historial de peticiones y se retiró entero horas después: todo el comportamiento errático giraba a su alrededor y tres arreglos sucesivos no lo dejaron estable. Un Repeater que mezcla respuestas es peor para un informe que un Repeater sin historial, y su lugar lo ocupa la decisión de diseño del apartado 9, una petición cada vez. La causa de fondo apareció después de retirarlo: el 302 invisible hacía que la respuesta del GET posterior pareciera la del POST.

El patrón de los dieciséis. Donde el Spider acumulaba fallos de traslado, aquí son de fidelidad: siete de los dieciséis mostraban algo distinto de lo que el producto hacía. En una herramienta de auditoría ese es el defecto más caro, porque lo que la herramienta muestra es la evidencia que acaba en el informe: un auditor que copie esas cabeceras documenta algo que no ocurrió.

El proyecto tenía un verificador configurado y sin usar. La regresión más tonta del tramo, el botón de enviar dejó de funcionar porque un borrado se llevó la declaración de una variable, la habría cazado el analizador estático del propio repositorio, que nunca se había ejecutado. Ahora va por delante de cada verificación y no queda ninguna variable sin declarar en el frontend. Lo que destapa de paso va al apartado 9: el frontend acumula veintitrés errores y dos avisos de otras reglas. Los defectos que el asistente introdujo aquí están en el apartado 11.

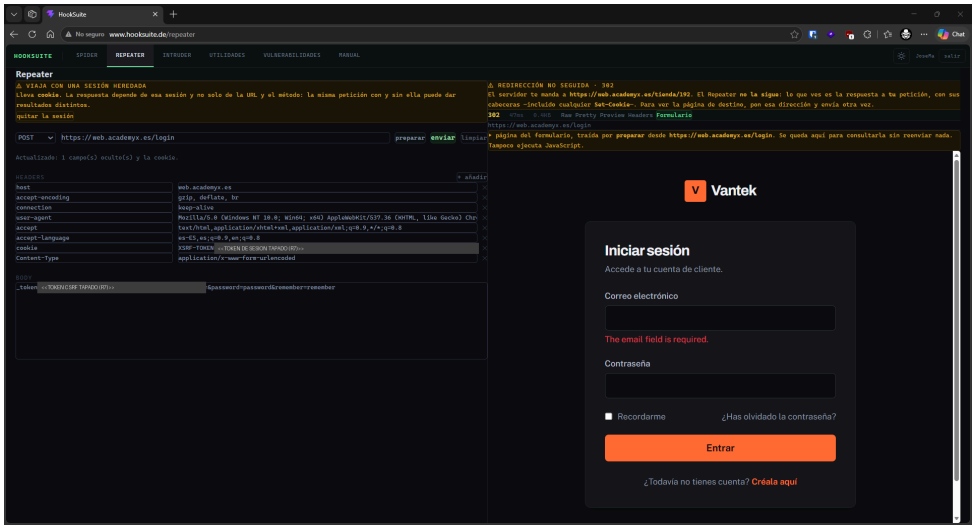


Figura 4: RF-05 · el Repeater tras pulsar «preparar» sobre un formulario traído del Spider: el aviso de sesión heredada con el botón que la retira, el acuse de campos ocultos y cookie actualizados, el aviso de redirección no seguida con el 302 y su destino, y la pestaña Formulario mostrando la página pedida sin reenviar nada. Los valores de sesión van tapados.

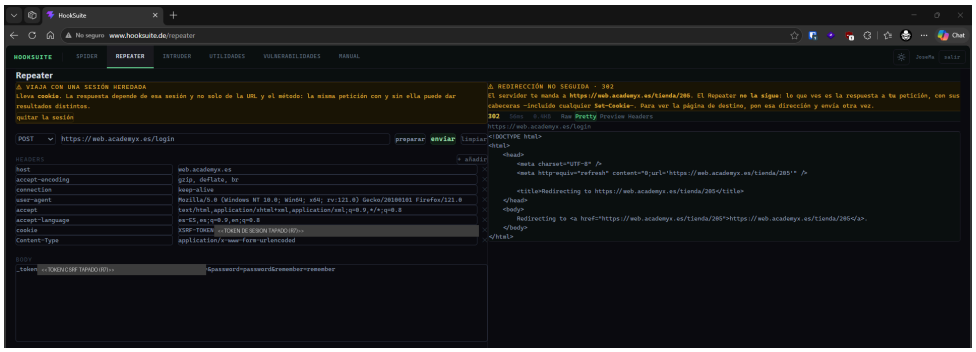


Figura 5: RF-04 · prueba de aceptación del Repeater con cliente propio, primer envío: 302, 0,4 KB y destino / tienda/205.

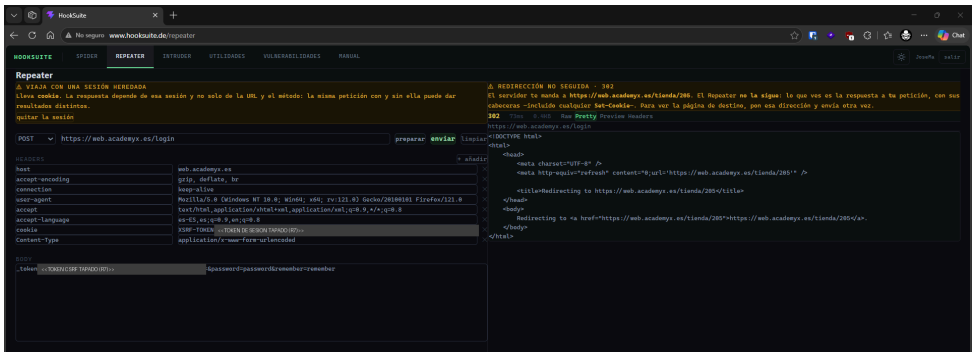


Figura 6: RF-04 · segundo envío de la misma petición, sin tocar nada: idénticos el estado, el tamaño, el destino y el cuerpo de la respuesta. Lo único que cambia es el tiempo (56 ms frente a 73 ms), y ese cambio es justamente lo que demuestra que son dos envíos distintos y no la misma figura repetida. Con el cliente compartido anterior, dos envíos iguales podían no coincidir.

> Nota de tratamiento de las figuras de este apartado. Se tomaron contra una web propia del equipo autorizada para ello. El panel de cabeceras muestra la cookie de sesión del objetivo, es lo correcto porque es lo que se envía, así que cada figura se publica con esos valores tapados y los originales no salen del almacén interno. En el par de aislamiento, la captura del usuario A llevaba el identificador de sesión capturado de la web auditada, justo el dato que RF-12 existe para no compartir: va tapado y con la etiqueta a la vista, para que se vea qué se redactó.

Intruder y Utilidades (RF-06, RF-07), tal como se entregan

Estas dos pantallas no se pulieron en esta práctica y se entregan como estaban, con un solo cambio en el Intruder, su cancelación corregida. Se comprueban igual: el Intruder con un ataque lanzado y cancelable, y el codificador resolviendo un JWT, que es la conversión que la verificación inicial había dado por ausente.

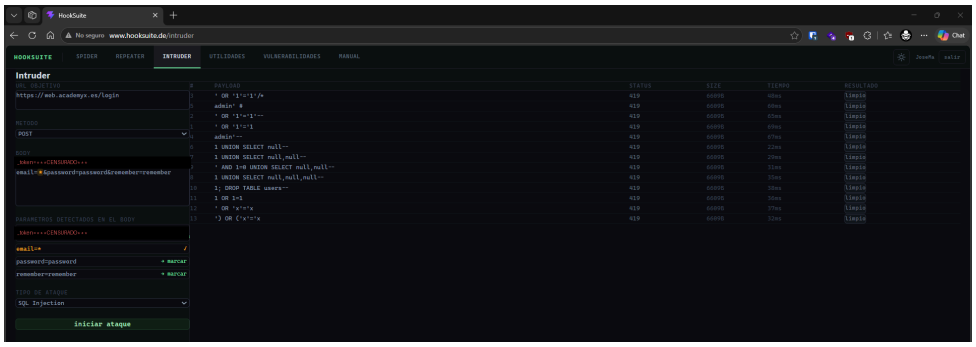


Figura 7: RF-06 · el Intruder con un ataque de inyección SQL lanzado contra el formulario de acceso del objetivo propio: trece payloads numerados, cada uno con su código, tamaño, tiempo y veredicto, y el parámetro marcado con el asterisco. Los trece responden 419 porque el objetivo exige un testigo CSRF válido en cada envío: lo que la captura demuestra es el mecanismo de fuzzing, no un hallazgo. El testigo va tapado.

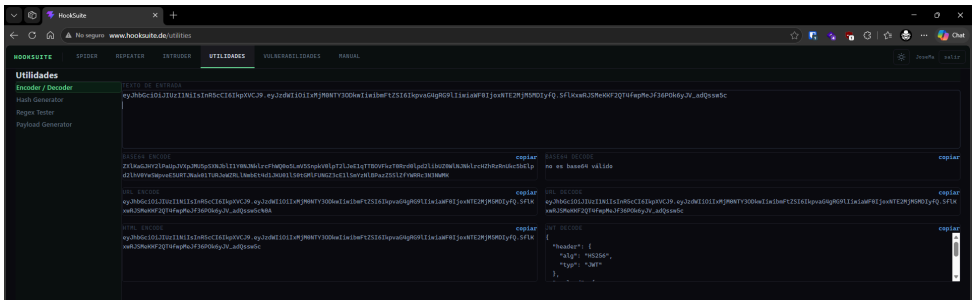


Figura 8: RF-07 · el codificador con un JWT en la entrada y sus cuatro conversiones resueltas a la vez: base64, URL, HTML y la lectura del JWT, justo la conversión que la verificación inicial dio por ausente. El testigo mostrado es el ejemplo público de documentación («John Doe»), no una credencial del producto.

El aislamiento (RF-12): las fugas capturadas antes, y la verificación después

Antes del arreglo, cuatro pruebas lanzadas contra el entorno local a través del proxy, porque el propio arreglo borra esta evidencia. Las cuatro confirmaron el fallo: la API responde 200 sin credencial, un token inventado obtiene sesión propia, un segundo usuario lee la cookie capturada por el primero, y una vulnerabilidad publicada sin token aparece en la sesión de dos auditores distintos. La captura íntegra, con la causa en fichero y línea de cada una, está en docs/evidencias/fugas-aislamiento-06oct.md.

Después, con el sistema de acceso puesto, se repitieron las mismas pruebas. Once comprobaciones, once en verde: alta y entrada de dos usuarios distintos, contraseña incorrecta rechazada con 401 y no con un error del servidor, la API respondiendo con token y 403 al intentar tocar el espacio de otro con token propio (autenticación y autorización, que son cosas distintas), y el segundo usuario sin ver ni el hallazgo ni la cookie del primero. Se probó además como ataque: un usuario autenticado intentó dirigir el Spider al espacio de otro nombrándolo en el cuerpo de la petición, el quinto fallo del apartado 6, y el rastreo fue a parar al espacio del atacante. Ni el código de invitación ni las contraseñas aparecen en ninguna salida: el guion de pruebas los mantiene en variables.

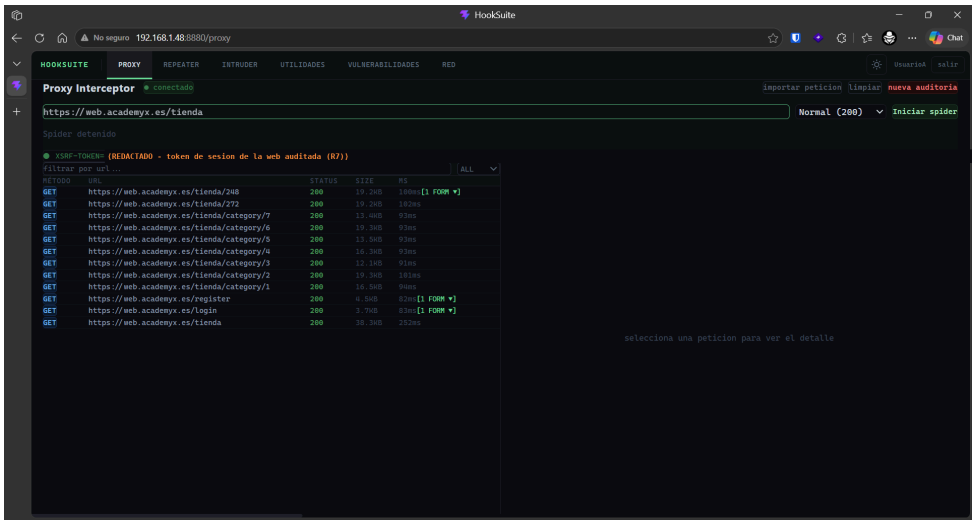


Figura 9: RF-12 · antes de rehacer el acceso este panel era común a todos. Ahora el usuario A, autenticado, ve las trece peticiones que su propio Spider capturó del objetivo autorizado. Compárese con la captura siguiente, tomada al mismo tiempo.

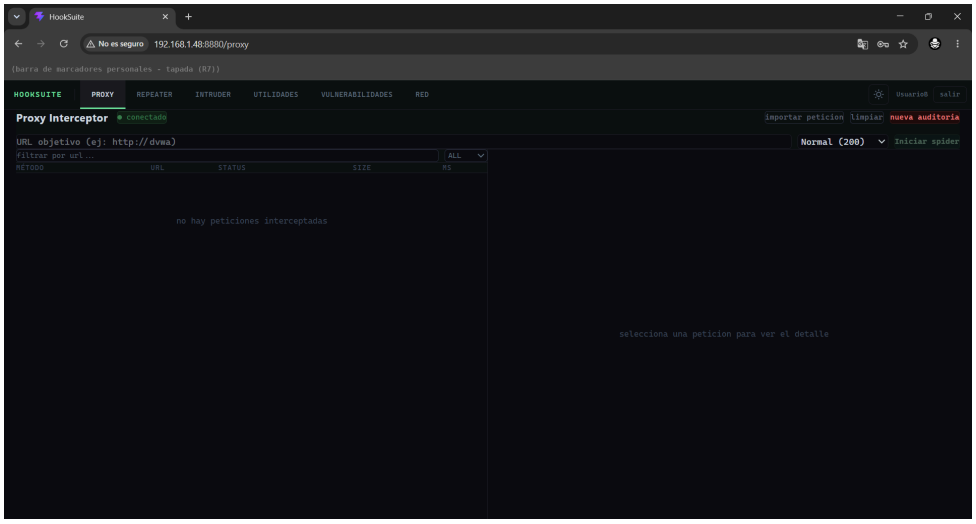


Figura 10: RF-12 · el contraste: el usuario B, autenticado a la vez y en otro navegador, ve su panel vacío. Antes del cambio habría visto exactamente lo mismo que el usuario A. Este par es la prueba del aislamiento por usuario. El alta de ambos exige el código de invitación: el registro no es abierto a propósito, porque la herramienta lanza tráfico contra terceros y con altas anónimas cualquiera podría atacar desde la infraestructura del grupo.

La batería de once casos de aislamiento pasa completa en el entorno local. En la instalación pública se han verificado las cuatro fugas cerradas, no la batería entera, y conviene no confundir una cosa con la otra.

La plataforma y el despliegue

Lo que falla cuando el producto se levanta, y no cuando se usa.

El despliegue (RNF-03) habría terminado «en verde» sin aplicar la configuración del proxy

La comprobación previa al despliegue del sistema de acceso encontró un defecto no en el producto, sino en el propio plan de despliegue. El plan mandaba reconstruir «solo el frontend, el backend y el proxy», pero el proxy no se construye: usa una imagen descargada y su bloque del fichero de composición no cambiaba ni un byte, de modo que Compose no lo habría recreado. Como el proxy lee su configuración una sola vez al arrancar, el despliegue habría

acabado sin errores y sin aplicar nada. Era el mismo fallo que ya había ocurrido dos días antes, y la razón de que se repitiera es instructiva: nombrar un servicio en un comando da la impresión de cubrirlo.

Después apareció el caso espejo. Se recrearon el frontend, la API y el módulo de IA y no el proxy, porque su configuración no había cambiado. Resultado: 502 en todo, porque los contenedores recreados reciben direcciones nuevas y el proxy resuelve los nombres de sus destinos una sola vez, al arrancar. Las dos mitades tienen la misma raíz, el estado del proxy se fija al arrancar, y de ahí sale la regla operativa que ahora está escrita: cuando se recrea cualquiera de los servicios a los que el proxy apunta, se recrea también el proxy, forzándolo explícitamente.

La instalación desde cero (RNF-03), ejecutada: dos defectos que solo aparecen al seguir las instrucciones

RNF-03 pide un despliegue reproducible, y lo único que lo respaldaba era que el servicio estaba en pie en producción, no que alguien pudiera levantarlo desde cero. La segunda cosa se probó siguiendo el README paso a paso, sobre un árbol independiente y con su propio proyecto de contenedores.

Salieron dos defectos, y ninguno se habría visto leyendo el documento. El primero: con el puerto 80 ocupado, el arranque falla con «address already in use» y el aviso del README ofrecía un único valor concreto como alternativa, que en esa misma máquina también estaba ocupado: seguir las instrucciones al pie de la letra producía el mismo error dos veces. El segundo es más fino, y es el que importa: ese fallo solo tumba el proxy, el único servicio que publica puerto, mientras los otros cinco arrancan con normalidad, de modo que quien no lea la salida del arranque ve cinco servicios en marcha y puede creer que ha instalado. Es la misma familia que el caso de arriba: algo que parece verde y no lo está.

El arreglo fue del documento, no del producto: el aviso deja de mandar a un número concreto, se añade cómo comprobar si un puerto está libre, y se advierte de que la comprobación final tiene que mostrar los seis servicios. Con eso la instalación llegó hasta la pantalla de acceso y se completó el alta con el código de invitación que genera el propio despliegue. Lo que esta prueba no es: la casilla del enunciado de que una persona ajena al grupo instale siguiendo el README, porque la condujo quien lo había escrito. Esa sigue abierta.

Defectos de montaje y arranque que impedían operar

Los tres aparecieron al reconstruir el entorno desde cero o al preparar un reinicio, y ninguno daba error visible.

- **El proxy dependía del laboratorio para arrancar**: infra/nginx.conf lo hacía requisito de arranque y el proxy fallaba con «host not found in upstream» si aquel no estaba levantado. Se retiró su ruta del proxy.
- **El navegador real estaba fuera de la red interna**: el servicio playwright no declaraba networks en el fichero de composición, la causa raíz de que RF-10 y RF-08 no llegaran a operar. Se corrigió con una línea y se verificó con docker compose config, que sitúa los seis servicios en la misma red.
- **Condición de carrera antes de reiniciar el servidor**: el agente del cortafuegos y el servicio de contenedores no tenían orden de arranque entre sí, y si el segundo ganaba, el agente quedaba en bucle de reinicios. Se corrigió fijando el orden antes de reiniciar, y tras el reinicio el contador del agente marcaba cero.

El módulo de análisis con IA, paso a paso

El recorrido de RF-08 se probó por tramos, y cada tramo dejó su propio fallo. Van en el orden en que se encontraron, del montaje a la respuesta del modelo.

Un módulo que parecía sano y no podía hablar con nadie (RF-08)

La prueba más barata de esta práctica fue una petición con curl, y descubrió un defecto que llevaba cuatro días en producción sin que nadie lo notara: el módulo de IA no podía comunicarse con el backend desde que se puso el control de acceso. El módulo preguntaba al backend cada cinco segundos si había trabajo, esa ruta quedó protegida junto con las demás, correctamente, y el módulo no presenta credencial ninguna, de modo que recibía un rechazo en cada intento.

Lo grave no es el defecto, es por qué no se vio. El código comprobaba si la respuesta era un éxito, y un rechazo por credencial no provoca ningún error, simplemente no es un éxito: el bucle volvía a dormir y el registro decía que estaba esperando instrucciones. Un operador habría visto un módulo en marcha y a la espera. Estaba en marcha, y no podía recibir nada. Es el mismo patrón que RNF-06 vino a cerrar un nivel más arriba, un fallo que no se distingue de la normalidad, y la razón por la que ahora el módulo falla ruidosamente: comprueba el bus al arrancar y se detiene si no está. Se midió con tres peticiones sin credencial, las tres rechazadas. Evidencia como texto en docs/evidencias/canal-ia-backend-muerto-10oct.md.

Las dos contrapartidas del bus (RF-08), probadas antes de confiar en él

Reconectar el módulo por un bus interno (apartado 4) cambia la forma del riesgo: un control de acceso falla cerrando, un bus falla abriendo. Así que lo que se prueba no es solo que el reparto funcione, sino que no se puede repartir mal. Dieciocho casos en dos conjuntos, con dobles para el bus, el gestor de sesiones y el orquestador: ni red, ni credencial de modelo, ni contenedores, cero coste.

Los dos que justifican el esfuerzo: un hallazgo sin dueño se descarta, no se guarda ni se emite (sin esa regla, un mensaje mal formado reabriría exactamente la fuga de aislamiento ya cerrada), y dos auditores no ven lo del otro, comprobado repartiendo a dos espacios y verificando que cada uno recibe solo lo suyo. Tres más cubren el fallo silencioso en el otro extremo: si la auditoría se interrumpe se publica el motivo, si nadie escucha el bus publicar lo dice en vez de dar el trabajo por encolado, y si una orden llega sin dueño no se audita. Reproducible con python3 backend/tests/test_bus_reparto.py y python3 ia/tests/test_bus_publicacion.py.

El disparador y el techo de gasto, verificados de punta a punta (RF-08)

Durante cuatro días el disparador estuvo construido y sin poder demostrarse: el botón existía, el bus estaba cableado, y una pulsación desde el panel no movía ni una línea del módulo de IA. Se cerró anotando la hora exacta de la pulsación y midiendo contra ella: a partir de las 13:58:10 el servidor registra una petición al disparador con respuesta correcta y el registro del módulo pasa de 192 a 376 líneas. Del panel a la autenticación, de ahí al bus y al módulo suscrito, el recorrido entero por primera vez. Lo que no se da por explicado: cincuenta y ocho minutos antes la misma pulsación no movió nada sin que se tocara el producto entre una y otra. Queda abierto y declarado, porque un apartado de pruebas que explica lo que no sabe explicar vale menos que uno que lo admite.

El mismo día se ejecutó el caso del techo de gasto. Una auditoría del módulo de IA anida páginas × tipos de prueba × cargas, y cada carga es una consulta al modelo: sin techo, cada pulsación arranca ese recorrido completo con gasto sin acotar. El caso estaba diseñado para costar cero: con el techo en 0 por variable de entorno, ninguna consulta debe salir y la auditoría

debe quedar marcada como truncada. Ejecutado desde el panel, con sesión real: la auditoría recorrió cuarenta y ocho elementos, no salió ni una sola consulta, coste 0 €, y los cuarenta y ocho quedaron marcados como no analizados, cada uno con el motivo del techo. El techo no se limita a cortar: el corte se ve, que era la condición que se le puso al diseñarlo.

La mitad que falta del caso, y se declara: el techo en 1, donde debe pasar una sola consulta y el resto quedar como «no analizado». Esa sí necesita clave de API válida, porque la consulta que pasa tiene que salir de verdad.

Dos cosas más quedan sin ejecutar y se declaran aquí: el caso del techo con límite distinto de cero, que necesita una clave de API válida de la que la instalación pública no dispone, y el caso formal con capturas de producto de dos defectos ya corregidos, el error de servidor de la comprobación de vida y la trampa de araña del rastreador, que están arreglados y verificados pero sin caso escrito.

Operación degradada (RNF-06): probada sin gastar saldo, y vista en el panel

La prueba que demuestra RNF-06 no necesita ni clave de API ni conexión: sustituye el cliente del modelo por uno simulado y comprueba qué devuelve el clasificador en ocho casos. Su valor está en haberla visto fallar: contra el código anterior devuelve *0 de 8* (los cuatro métodos llamaban a .get() sobre un objeto que ya no existe y la fase de análisis se interrumpía entera) y contra el corregido *8 de 8*. Una prueba que pasa siempre no prueba nada. Reproducible con python3 ia/tests/test_tres_estados.py, evidencia con las dos salidas en docs/evidencias/rnf06--tres-estados-antes-y-despues-09oct.md.

Una prueba unitaria demuestra que el clasificador devuelve el estado degradado, no que alguien lo enseñe, y entre las dos cosas hubo un hueco real: durante un día el estado existió y no lo consumía nadie al otro lado, de modo que RNF-06 estaba cumplido y era invisible. El hueco se cerró y se comprobó en la interfaz, con una auditoría lanzada desde el panel y sesión real.

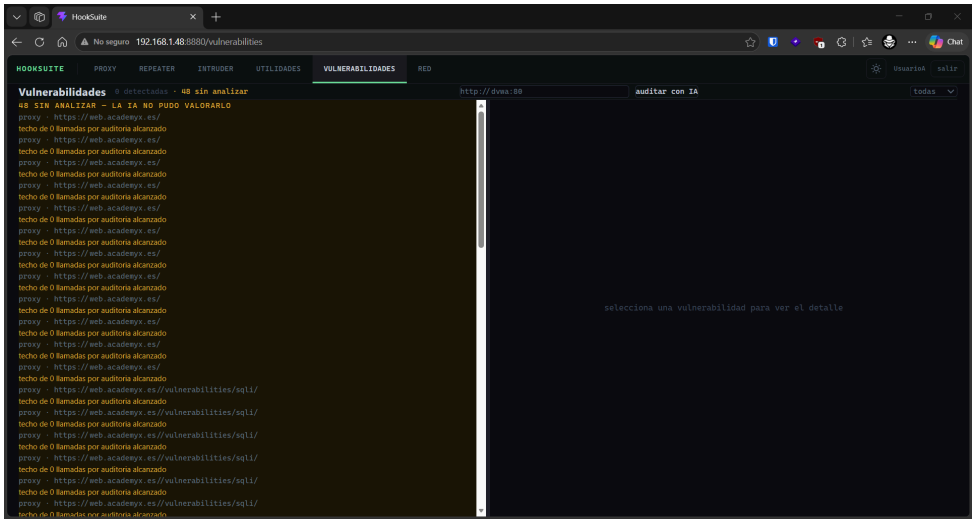


Figura 11: RNF-06 / RF-08 · el panel tras una auditoría truncada por el techo de gasto: 48 sin analizar, la IA no pudo valorarlo, cada entrada con su motivo («techo de 0 llamadas por auditoria alcanzado») y el recuento «0 detectadas · 48 sin analizar» separando las dos cosas. Los elementos provienen de un rastreo previo sobre una web propia del equipo. El objetivo de la barra era el valor fijo que el panel traía entonces, sustituido después por la herencia del objetivo rastreado.

Lo que la figura prueba, y que ninguna prueba unitaria alcanzaba: que el mensaje de lista vacía dejó de mentir. Antes, una auditoría que no había podido analizar nada y otra que había analizado y no encontrado nada producían la misma pantalla. Ahora el aviso va aparte de los

hallazgos, en su propia tira, por encima, porque mezclarlo obligaría a inventarle una severidad a algo que precisamente no se pudo valorar.

Y volvió a verse, en producción y por un motivo distinto. Una auditoría sobre la instalación pública recorrió cuarenta y ocho elementos y ninguno llegó a analizarse, con dos motivos conviviendo en la lista: «clave de API invalida o ausente» en los intentos que salieron y «techo de 40 llamadas por auditoria alcanzado». Tiene más valor que el caso de laboratorio porque no se provocó: la vía degradada se activó por una condición real del entorno, el cliente la devolvió como estado y no como excepción, y la pantalla la mostró separada de los hallazgos y con su motivo. El mismo mecanismo, dos causas distintas: eso es lo que RNF-06 pedía.

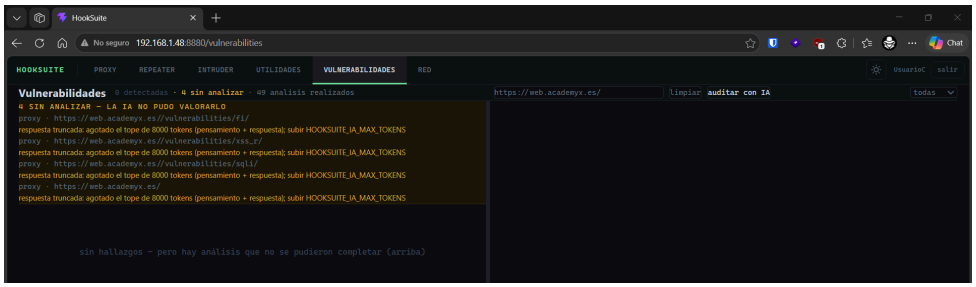


Figura 12: RF-08 / RNF-06 · la tercera forma de quedarse sin análisis: cuarenta y nueve análisis realizados, cero hallazgos y cuatro elementos sin analizar porque la respuesta se truncó al agotar el tope de tokens. El mensaje en pantalla recomienda subir ese tope y el consejo es erróneo: el cliente no pide la respuesta por partes, de modo que subirlo hace fallar la consulta. La barra de pestañas es anterior a la retirada de la pestaña Red.

Lo que el modelo puede devolver, y lo que el código daba por hecho (RF-08)

Las cuatro vías de análisis del módulo de IA piden al modelo una respuesta con un formato fijo, impuesto mediante un esquema que viaja con la consulta: la API garantiza que la respuesta se ajusta a él. El esquema no es documentación, es el control, y un campo declarado con tipo abierto no es un campo permisivo, es un campo sin control. La prueba comprueba que el esquema obliga a lo que las instrucciones piden, que la bandera de resultado sea un booleano y que la confianza venga en escala de 0 a 100. Contra el código anterior devuelve *5 de 11*, y los fallos no eran cosméticos:

1. De los tres campos de bandera, uno por vía, solo el primero estaba declarado como booleano. Los otros dos admitían cadena de texto, y el clasificador evalúa los tres dentro de un if: en el lenguaje en que está escrito, cualquier cadena no vacía es verdadera, incluida la palabra que significa «no». Una respuesta negativa del modelo se habría registrado como vulnerabilidad confirmada.
2. La confianza se declaraba como número sin rango, así que nada rechazaba un valor fuera de la escala.
3. El esquema de la vía de identificación de servidor declaraba cero campos, mientras sus instrucciones piden nueve.

La causa era una sola y estaba fuera del repositorio del producto. Los esquemas los genera una herramienta de la práctica cuya lista de campos, escrita a mano, se redactó mirando una sola de las cuatro vías. El esquema vacío tiene la misma raíz por el camino opuesto: esa vía devuelve la respuesta sin inspeccionarla, y un esquema no se puede derivar de un código que no mira los datos. El arreglo fue en el generador, no en el fichero generado: la prueba pasa *11 de 11* y la de RNF-06 sigue en *8 de 8*. Un matiz queda como limitación conocida: la API no admite mínimo ni máximo en salida estructurada, de modo que el rango 0-100 tuvo que salir del esquema y un valor fuera de escala ya no lo para nadie. El caso peligroso, el de la escala 0-1, lo cubre la guarda del clasificador de más abajo.

Reproducible con python3 ia/tests/test_esquemas_bandera.py ia/esquemas.py. Evidencia con las dos salidas, antes y después, en docs/evidencias/esquemas-bandera-antes-10oct.md.

Tres rechazos seguidos de la API: validar contra tu propia prueba no es validar (RF-08)

La primera auditoría real, clave válida, objetivo propio, techo de gasto puesto, no devolvió ni un hallazgo: las llamadas que salieron se rechazaron antes de procesarse, y hubo que lanzarla tres veces porque cada rechazo tenía un motivo distinto. Los objetos del esquema iban abiertos, y la salida estructurada no lo admite. La confianza declaraba un rango con mínimo y máximo, que tampoco se admite. Y veintidós campos declaraban su tipo como una lista de tipos posibles que incluía «objeto», lo que exige una declaración explícita que esos campos no llevaban. Coste: cero, porque un rechazo por petición mal formada no consume saldo. La factura fue de tiempo.

Lo que este episodio enseña. La prueba de esquemas daba *11 de 11* y las seguía dando con el defecto dentro: comprobaba lo que nosotros le pedíamos al esquema y nunca lo que la API exige para aceptarlo. Dicho corto: un esquema validado contra tu propia prueba no está validado. Está de acuerdo contigo. Quedan dos piezas para que no haya una tercera vez: ia/tests/test_esquemas_api.py recorre los esquemas y rechaza cualquier clave que no conste como admitida (verificada distinguiendo las dos versiones, *1 de 5* contra los defectuosos y *5 de 5* contra los corregidos), y ia/tests/validar_esquemas_contra_api.py somete cada esquema al servicio real, los cuatro pasan, porque cuando el contrato lo pone otro, se le pregunta a él. Los dos defectos nacieron en el mismo sitio, el generador, la única pieza del proyecto sin control de versiones: lo que no tiene historial es donde nadie mira.

Un fallo silencioso no es un fallo menor: la escala de la confianza (RF-08 / RNF-06)

El filtro por confianza compara el valor que devuelve el modelo contra un umbral. Las instrucciones piden ese valor en escala de 0 a 100, pero ninguna pide que sea entero, y ahí había una rendija que el esquema no puede cerrar: un 0,85 queriendo decir «85 %» está dentro del rango permitido, pasaría el control y quedaría por debajo del umbral. El hallazgo desaparecería sin dejar rastro, que es exactamente lo que RNF-06 existe para evitar. El arreglo va en el clasificador y no en el esquema, porque imponer «entero» habría rechazado un decimal legítimo: la franja entre 0 y 1 se trata como sospecha de escala y se encamina al estado degradado de RNF-06, con su motivo, de modo que el problema se ve en el informe en lugar de desaparecer.

La prueba distingue las dos versiones, que es la única forma de que valga: contra el código corregido devuelve *19 de 19* y contra el anterior *6 de 19*, y los seis que pasan en la versión antigua son precisamente los casos legítimos, lo que demuestra que la guarda no introduce falsos positivos. Reproducible con python3 ia/tests/test_escala_confianza.py. Evidencia con las dos salidas en docs/evidencias/escala-confianza-guardia-10oct.md.

SECCIÓN 8

Matriz de trazabilidad

La pieza con la que se corrige la práctica: requisito por requisito, dónde está implementado, qué prueba lo verifica y dónde se ve. Las rutas están comprobadas en el código. La columna de evidencia enlaza la figura de la memoria y el apartado donde se explica.

Req.	Descripción	Estado	Implementación	Prueba y evidencia
RF-01	Panel web sin instalación	Cumplido · seis pestañas	App.jsx + Layout.jsx + pages/manual/	PR-01 · Figura 13
RF-02	Proxy del lado servidor (httpx)	Modificado · apdo. 3	proxy_service.py	PR-19 · apdo. 3
RF-03	Spider / rastreo	Cumplido · 19 defectos corregidos	spider_service.py + pages/proxy/	PR-03 · Figura 3
RF-04	Auditoría autenticada	Modificado · cliente propio, apdo. 3	session_service.py + proxy_service.py	PR-04 · Figura 5
RF-05	Repeater	Cumplido · 16 defectos, sin historial por diseño	repeater.py + pages/repeater/	PR-05 · Figura 4
RF-06	Intruder / fuzzing	Cumplido · cancelación corregida	intruder_service.py	PR-06 · Figura 7
RF-07	Utilidades (incl. JWT)	Cumplido	pages/utilities/	PR-07 · Figura 8
RF-08	IA que alimenta Vulnerabilidades	Parcial · recorrido y techo probados, ningún hallazgo analizado	ia/ + redis_consumer.py	PR-08 · Figura 11
RF-09	DevTools que alimenta Red	Descartado · apdo. 3	devtools/ y network.py conservados	apdo. 3
RF-10	Playwright	Parcial · sin prueba ejecutada, sondeo con 401	playwright/ + routes/ playwright.py	PR-10 · apdos. 6-7
RF-11	Multiusuario	Cumplido · rehecho con el acceso por usuario	guardia.py + auth_service.py	PR-11 · apdo. 6
RF-12	Registro, login y aislamiento	Cumplido · fugas cerradas, 11/11 en local	auth_service.py + guardia.py	PR-12 · Figura 9
RNF-01	Acceso sin instalar nada	Cumplido · con limitación en móvil	frontend/ tras Nginx	PR-01 · apdo. 9
RNF-02	Tiempo real (WebSocket)	Cumplido · canal autenticado	main.py (/ws/)	PR-17 · Figura 2
RNF-03	Despliegue reproducible	Cumplido · probado desde cero	docker-compose.yml + nginx.conf + README	PR-13 · apdo. 7
RNF-04	Historial de commits del grupo	Parcial · sin reparto, autoría sorteada	repositorio del producto	apdo. 10
RNF-05	Límite de concurrencia	Cumplido	intruder_service.py	PR-20 · apdo. 7
RNF-06	Operación degradada	Cumplido · visto en el panel	client.py + analyzers/	test_tres_estados (8/8) · apdo. 7
RNF-07	Seguridad del producto	Parcial · dependencias con avisos sin actualizar	ver apdo. 6	PR-14 y PR-16 · apdos. 6-7
RNF-08	Cortafuegos por sesión	Cumplido en la P1 · no reejercitado	firewall_agent.py	PR-15 · apdos. 6-7
RNF-09	TLS/HTTPS	Pendiente	por configurar en Nginx	PR-18 · apdo. 7

La columna de prueba remite al plan de pruebas del apartado 7, que reconstruye los dieciocho casos citados con dos resultados, el del prototipo y el de hoy sobre el código que se entrega, y declara caso por caso de dónde sale el resultado de hoy. La numeración RF/RNF es la fijada en el apartado 3.

SECCIÓN 9

Limitaciones y trabajo futuro

Qué no funciona, qué funciona con condiciones y qué se haría con más tiempo. Este apartado recibe material hasta el último día.

El estado de sesión es volátil, por diseño

El estado de una sesión de auditoría vive en memoria del backend: al reiniciarlo, se pierde. Es una decisión consciente: persistirlo obligaría a reescribir la treintena de puntos del backend que mutan la sesión en memoria, a pocos días de la entrega, y una sesión de auditoría es un espacio de trabajo efímero del que el operador exporta lo que importa. Lo que sí se hizo, porque era el riesgo real, es poner topes: el recolector de sesiones existía y nunca se invocaba, y cualquier llamada con un identificador nuevo creaba una sesión más, lo que permitía agotar la memoria del servidor. Ahora hay tope de sesiones y de peticiones por sesión, el recolector se ejecuta y al desalojar prefiere las sesiones sin conexión abierta. Verificado con prueba de abuso: 120 identificadores inventados dejan el número de sesiones en el tope y la sesión con conexión viva sobrevive. Trabajo futuro: la persistencia es viable sin cambiar la arquitectura, el despliegue ya incluye Redis.

El techo de gasto de la IA: aplicado, y lo que sigue sin estar acotado

Lo que hay hoy. El cliente del modelo lleva un techo de consultas por auditoría, situado ahí porque por el cliente pasan las cuatro vías de análisis. Su valor por defecto es de cuarenta consultas, configurable por entorno, y al agotarse devuelve la vía degradada de RNF-06: la auditoría queda marcada como truncada en vez de parecer completa. Se comprobó en producción que el recorte llega hasta la pantalla con sus propias palabras, elemento por elemento. Lo que sigue siendo limitación: el techo cuenta consultas y no dinero, y lo hace por auditoría, de modo que acota el caso peor de una y no el gasto de una jornada. Y el valor de cuarenta entró como provisional y sigue siéndolo: no se ha reajustado.

Lo que queda de la confusión de escala, una vez cerrada

Lo que sí queda, y es consciente:

1. Un valor legítimo de 1 se marca como sospechoso. La guarda no puede distinguir «1 %» de «certeza total en escala de 0 a 1», porque son el mismo número. El coste es un aviso de más y la ganancia es que el caso peor, descartar un hallazgo seguro, deja de ser posible: preferir el falso positivo visible al falso negativo silencioso es la misma elección que ya tomó RNF-06.
2. La vía de identificación de servidor no lleva guarda, porque no compara contra el umbral: no hay descarte silencioso que cerrar.
3. No se cambiaron las instrucciones del modelo para pedir un entero, que era la alternativa más limpia: son cuatro ficheros del material que no se vuelca a la conversación y se editarían sin verlos, con el desarrollo a punto de cerrarse. Queda como trabajo futuro de coste bajo.

El manual de uso va por dos pestañas escritas y tres por escribir

Se añadió una sexta pestaña, **Manual**, con el modo de uso de cada herramienta. Ataca de raíz el patrón descrito en el apartado 7, funciones correctas que no se ven, y por eso no es documentación accesoria sino parte del producto.

Están escritas dos secciones de cinco: la del Spider, con los veinticinco controles de esa pantalla detallados uno a uno (tipo de elemento, qué hace y bajo qué condición aparece), y la del Repeater, con veintiséis. Las tres secciones sin escribir se marcan a la vista del usuario: si falta, se ve que falta.

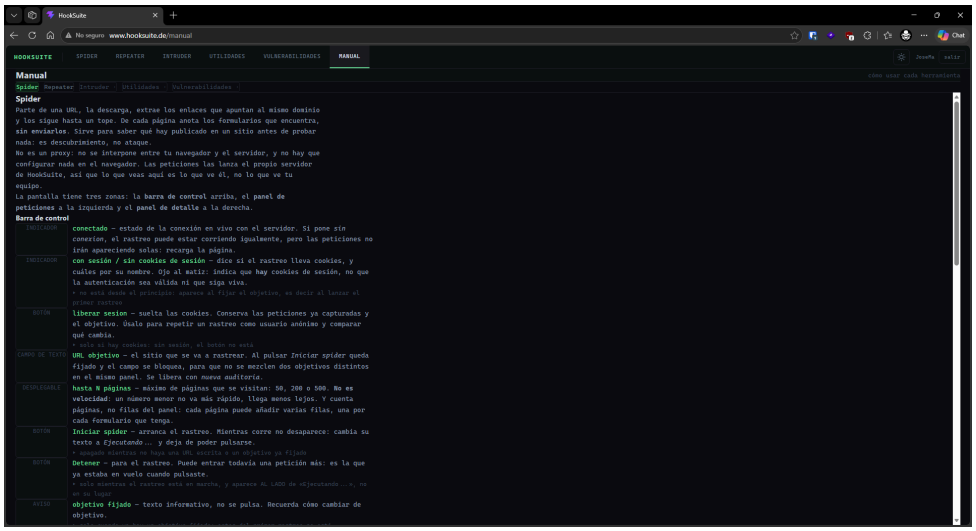


Figura 13: RF-01 · la pestaña Manual: cinco pastillas, la activa resaltada y las tres secciones sin escribir marcadas con un punto, de modo que el usuario ve que el manual está incompleto. Debajo, el modo de documentar: cada control con su tipo (indicador, botón, campo de texto, desplegable, aviso), qué hace y bajo qué condición aparece.

La regla fijada es deliberada: una sección se escribe cuando su pestaña se ha pulido y verificado, nunca antes, porque durante el pulido del Spider lo que suponíamos de la pantalla y lo que el código hacía no coincidieron quince veces, y un manual redactado sobre suposiciones induce a error. La propia redacción destapó tres defectos más, entre ellos el de los filtros (apdo. 7) y veinte líneas de código muerto: documentar obliga a mirar el código con los ojos de quien lo usa. Queda pendiente decidir si el manual lleva capturas propias, tomadas para enseñar a usar y no para demostrar un arreglo.

Dos cosas del Spider que se dejan dichas, no hechas

- **Cuántas veces aparece un formulario.** Un formulario se cataloga una sola vez, bajo la primera página donde se encontró. Es lo correcto para no inflar el recuento, pero se pierde un dato útil para el auditor: en cuántas páginas está presente. La etiqueta actual dice «vistos aquí primero», que avisa del comportamiento sin dar el número.
- **La etiqueta del desplegable.** Se corrigió para que no insinuara más de lo que hay, y quedó anotada para revisarla en el rediseño de la interfaz previsto como trabajo futuro.

El Repeater trabaja con una petición cada vez, y es una decisión

El Repeater no tiene historial: mantiene una petición cargada y, mientras la tenga, el Spider no permite enviarle otra, su botón pasa a «repeater en uso» y explica cómo liberarlo. Es lo primero que echará en falta quien conozca la herramienta de referencia, y se sostiene como diseño y no como carencia: el historial existió durante nueve horas y se retiró (apartado 7) porque el desorden venía de tener varias peticiones conviviendo sin que nada dijera cuál era la buena. Para una herramienta cuya salida acaba en un informe, que no se pueda confundir qué respuesta pertenece a qué petición vale más que poder recuperar la de hace diez minutos. Trabajo futuro, con el camino ya identificado: una lista propia en la sesión del servidor, separada de la que comparten el Proxy y el Spider.

El borrador del Repeater guarda las credenciales del objetivo en el navegador

Limitación conocida y declarada. El editor del Repeater guarda lo que estás escribiendo en el almacenamiento de sesión del navegador, para que no se pierda al cambiar de pestaña, y

ese guardado no filtra las cabeceras de sesión: la Cookie del sitio auditado, credenciales de un tercero, queda escrita en el navegador del auditor. El recorrido es la parte aprovechable: esa protección se añadió en la persistencia del historial, y al retirar el historial esa misma tarde la función de filtrado se fue con él, mientras que el borrador nunca pasó por ese filtro. El patrón que conviene retener: una medida de seguridad acoplada a la función que la motivó se pierde cuando esa función se retira, y no lo detecta ninguna prueba, porque el producto sigue funcionando igual.

Riesgo real, acotado sin minimizarlo. El almacenamiento de sesión es legible por cualquier código que se ejecute en la página, de modo que aprovecharlo exige antes una ejecución de código en el propio panel. La única superficie de ese tipo, la vista previa de la respuesta del objetivo, está aislada, y el almacenamiento se vacía al cerrar la pestaña. No es una puerta abierta: es un dato que no debería estar ahí. La corrección son tres líneas, reutilizar al guardar la lista de cabeceras de sesión que el editor ya usa para avisar, y se decidió no tocar el producto con el desarrollo ya cerrado.

Lo que falta para igualar al Repeater de referencia

Tres diferencias conocidas y no cerradas:

- **No hay botón para seguir una redirección.** El Repeater muestra el 3xx y no lo sigue, que es lo correcto. La herramienta de referencia ofrece además seguirla con un clic: trabajo de interfaz y de un extremo nuevo, fuera por la fecha.
- **El origen de las peticiones, que es la diferencia de fondo.** En la herramienta de referencia el formulario llega capturado del navegador, una petición que ya funcionaba. Aquí llega una plantilla leída del HTML por el Spider, que nunca se envió. El botón «preparar» cierra la distancia a mano, pero eliminarla exige capturar el tráfico real del navegador, que es justamente lo que se descartó con RF-09.
- **Las cabeceras repetidas se muestran unidas por coma**, no como dos filas. Se ven, antes se perdía una, pero no se editan por separado.

El frontend no pasa su propio analizador estático

El repositorio tiene configurado un analizador estático para el frontend y nunca se había ejecutado (apartado 7). Puesto a funcionar, arroja veintitrés errores y dos avisos que no son del trabajo de estos días, casi todos de reglas del marco de interfaz y concentrados en la pantalla de vulnerabilidades. Lo que sí quedó cerrado es la regla que caza el fallo de la regresión del botón de enviar: no queda ninguna variable no declarada en todo el frontend. No se arreglan ahora porque son avisos de calidad sobre pantallas que funcionan y ya están capturadas, y tocarlas a estas alturas arriesga el entregable sin mejorar el producto: queda como trabajo futuro con el dato ya medido.

Restos del laboratorio retirado

El laboratorio vulnerable salió del producto (apartado 6), pero su nombre sigue en dos sitios que no se ejecutan: la función de acceso del módulo de navegador (RF-10, parcial) y los datos de ejemplo del frontend, que sí viajan en el paquete que se sirve al navegador aunque solo se usen cuando no hay servidor detrás. No es un riesgo, los valores son inventados, pero el entregable menciona un componente que ya no incluye. Es limpieza de diez minutos que no se hizo a propósito: tocar código que funciona, el último día, para arreglar un nombre, es justo el cambio que no compensa.

La herramienta que genera los esquemas no está versionada

El generador de esquemas vive en la carpeta de trabajo de la práctica, fuera del repositorio del producto: sin historial, sin revisión y sin autoría registrada, al contrario que todo el código. No es una incoherencia inocua: el único sitio sin historial resultó ser exactamente donde estaba el defecto del apartado 7. Trabajo futuro de coste bajo: mover las herramientas al repositorio, cosa que no se hizo con el desarrollo a punto de cerrarse porque introducía riesgo sin aportar nada al producto.

El vídeo de demostración no se entrega

De los tres entregables que pide el enunciado se entregan dos, la memoria técnica y el repositorio. El vídeo de demostración no se ha grabado por falta de tiempo en la última semana de la práctica, y queda declarado aquí en lugar de entregarse sin explicación.

Qué se pierde con ello. El enunciado reserva al vídeo la demostración del producto en marcha y pide que cada requisito lleve anotado en la matriz el minuto donde se demuestra. Quedan sin cumplir las tres comprobaciones de la lista de entrega que dependen de la grabación y la que pide el enlace en la portada, y la matriz se entrega sin columna de minutos.

Qué queda en su lugar. Las figuras del producto citadas una por una en la matriz de trazabilidad, el plan de pruebas del apartado 7 con sus dieciocho casos y sus dos resultados, y la instalación desde cero ejecutada y documentada en el apartado 7. Ninguna de las tres sustituye al vídeo y no se presentan como equivalentes.

SECCIÓN 10

Reparto del trabajo

Quién hizo qué, con estimación de horas por persona.

Cómo se repartió el trabajo, y por qué el historial no lo refleja

El producto es de los cinco. HookSuite lo diseñó y lo construyó el grupo entero en la Práctica 1, interfaz, servidor, navegador real, captura de tráfico y módulo de IA, un módulo por persona, y ese es el producto que se entrega aquí. La Práctica 3 no construye una herramienta nueva: coge la de la P1 y la lleva a un estado ejecutable, seguro y reproducible. Todo lo que esta memoria cuenta descansa sobre el trabajo de los cinco.

El plan de trabajo también es del grupo. El alcance se decidió en la reunión inicial: abordar los cinco primeros puntos del plan de mejoras que la propia P1 dejó escrito. El plan desarrollado se presentó al grupo y recibió el visto bueno con tres reparos, atendidos uno a uno. El orden del trabajo y lo que se descarta salen de ahí.

Lo que no se pudo hacer en grupo fue la ejecución. Entre el trabajo de cada uno, las clases del máster y el día a día, al grupo le resultó imposible trabajar de forma presencial o continuada durante estas semanas. La práctica se ejecutó en sesiones de trabajo con la ayuda de un asistente de IA, el apartado 11 lo declara con detalle: todo el código añadido o modificado en esta práctica lo escribió el asistente, con el plan del grupo como guía y consultas al grupo en las decisiones de alcance.

Y las confirmaciones del repositorio no indican quién escribió cada línea. Se reparten al azar, a propósito, y lo que viene a continuación explica el mecanismo y su única excepción. Se declara aquí porque el enunciado dice expresamente que el historial de confirmaciones es una de las fuentes con las que se juzga el reparto: en este caso, usarlo para eso daría una foto equivocada.

Y lo que esta memoria no va a decir, aunque sonaría mejor: que el grupo pudo trabajar de forma fluida y repartida en esta práctica. No ocurrió. Escribirlo convertiría una limitación de calendario, comprensible, ajena al producto y que no cambia lo que la herramienta hace, en una afirmación falsa sobre el equipo. Una confirmación con un nombre no es una afirmación de que esa persona escribiera ese código, y no debe leerse así.

El mecanismo del sorteo de confirmaciones

La autoría de cada confirmación se asigna al azar entre las cinco personas del grupo, y se declara aquí porque el historial no dice quién hizo qué. En esta práctica no hubo un reparto de tareas que reflejar: las decisiones se fueron tomando en consulta de grupo continua y la ejecución pasó por una sola vía, de modo que ninguna confirmación es atribuible a una persona, y firmarlas todas a un mismo nombre habría dado una foto igual de equivocada sobre un producto que es de los cinco. Se eligió el sorteo, y se dice que es un sorteo. Con una excepción fija: la memoria técnica se firma siempre a nombre de la misma persona, porque es un documento único que se regenera entero en cada cambio.

Esa excepción deforma el recuento, y el dato concreto lo demuestra: de las 41 confirmaciones de una sola jornada, una persona figura en 19, más del doble que cualquier otra, pero 11 de esas 19 son regeneraciones de la memoria. Descontada la excepción quedan 8, que es justo lo que cabe esperar de un sorteo entre cinco sobre 41. El reparto es uniforme. Lo que no es uniforme es el documento. Dos correos del mismo autor parten además el grafo de contribuciones de GitHub, y se declara aquí como parte de RNF-04. La estimación de horas de este apartado sale del diario, no del historial de confirmaciones.

Horas registradas: qué parte es medida y qué parte inferida

El diario tiene 179 entradas, 177 con campo de horas, y suma 87,2 h. El reparto de abajo separa a propósito las tres cosas que suelen ir mezcladas en una tabla de horas: lo que está escrito, lo que está **inferido con una regla declarada**, y lo que simplemente no se registró.

Quién	Horas	De dónde sale esa cifra
Claude Code (asistente)	**87,1 h**	**40,7 h escritas** con su nombre en el campo de horas, todas del tramo final de la práctica · **46,4 h inferidas**: entradas anteriores cuyo campo dice solo « 1 h», sin nombrar a nadie
José María	**≈1 h de la reunión, más guía continua no cronometrada**	La reunión inicial, como el resto del grupo, y desde ahí la guía de la ejecución sesión a sesión, que no se cronometró porque va entremezclada con el uso de la herramienta. Lo que sí se puede contar son sus intervenciones: **43 entradas** las registran (23 decisiones, 5 revocaciones de claves, 5 pruebas en producción, 5 lotes de capturas, 3 ejecuciones en la instalación pública y 2 fusiones)
Ivan · Macarena · Nacho · Carlos	**≈1 h cada uno, más consulta continua no cuantificable**	La **reunión inicial**, de una hora y con los cinco. Desde ahí, **consulta de grupo continua** en mensajes sueltos sobre la herramienta y cada duda: es real, pero no se puede cronometrar ni repartir por

		persona, y aquí no se inventa un número. Lo que no hubo fue reparto de tareas

La regla de inferencia, dicha para que se pueda discutir. En ese tramo final el campo escribe « 0,5 h de Claude». Antes escribía « 1 h» a secas. Las entradas las redactó el mismo autor de principio a fin, con el mismo formato y el mismo propósito, medir el tiempo de la sesión, así que se infiere que el campo significaba lo mismo antes de decirlo. No es una medida y no se presenta como tal: son 46,4 de las 87,2 h, más de la mitad, y por eso la cifra del asistente se da como 87,1 h con su desglose a la vista en lugar de como un total redondo. (40,7 + 46,4 = 87,1. La décima que falta hasta 87,2 es redondeo de cada entrada.)

Lo que esta tabla no es. No es el esfuerzo del grupo en el producto, que los cinco escribieron en la práctica anterior: el diario solo cubre las sesiones de la P3. Tampoco mide valor: una decisión de treinta segundos cambió el entregable más que varias horas de construcción. Y el límite honesto: fuera de esa hora de reunión, el tiempo del grupo no está en ningún registro. No se cronometró, y aquí no se inventa.

SECCIÓN 11

Uso de herramientas de IA

Declaración obligatoria de qué herramientas de IA se usaron y para qué.

- **Claude Code** (modelo Claude de Anthropic): asistencia en diagnóstico, montaje del entorno local de pruebas, ensayo de limpieza del historial y redacción de esta memoria, bajo revisión humana.

Los límites de la herramienta, observados en primera persona

Usar un asistente de IA para auditar seguridad tiene un límite que encontramos trabajando: el filtro de seguridad del propio modelo bloquea a veces tareas legítimas de este dominio. Leyendo las instrucciones del clasificador de vulnerabilidades de HookSuite, la petición fue rechazada con un error que lo dice con todas las letras, el filtro «a veces marca tareas legítimas de programación, ciberseguridad y biología». Es el coste de trabajar en seguridad ofensiva con una herramienta generalista, y se cuenta con la fecha delante en lugar de omitirlo.

Lo caro no fue el bloqueo, sino la recuperación: la opción elegida del rebobinado de la herramienta descartó ocho minutos de trabajo que otras dos opciones habrían conservado. Y de ahí salió el hallazgo que sí afecta al diseño de nuestros controles: el rebobinado restaura la conversación y los ficheros, pero no los efectos sobre el servidor. El turno que había construido una imagen de contenedor figuraba como «sin cambios de código», la imagen siguió existiendo después de rebobinar, y el único testigo fue un control externo que compara el estado del servidor contra lo que la documentación afirma. Un control que mira ficheros no sustituye a uno que mira el sistema.

Qué fue generado y qué fue verificado. La distinción no es teórica: ese mismo día, una afirmación escrita por el asistente en el plan del módulo de IA, que los cuatro conjuntos de instrucciones pedían un campo descripcion, resultó falsa para uno de ellos al releer el código fuente, y se corrigió. El criterio que seguimos es que nada entra en esta memoria por haberlo

dicho la herramienta: entra cuando se ha contrastado contra el código, la salida de un comando o el enunciado.

Los defectos que introdujo el asistente al arreglar otros

Dos de los quince defectos del pulido del Spider (apdo. 7) los causamos nosotros, y la cuenta llegó a ocho en una sola jornada. Todos comparten la misma forma: un arreglo correcto en lo suyo que rompe algo contiguo que nadie volvió a mirar. Lo que importa no es el número, sino qué los detectó:

Nº	Qué rompió el asistente	Qué lo detectó
1	Los escapes de un aviso nuevo se mostraban en crudo en la pantalla	los ojos de una persona leyéndolo
2	La dirección de destino se colocó en una barra sin sitio: solo se leía cuando la petición fallaba	una captura de producto
3	Un comentario colocado entre los atributos de un componente rompió la construcción	la construcción de la imagen
4	El arreglo de las cabeceras del Spider se aplicó a uno de los dos sitios, y todos los POST empezaron a dar 419	usando el producto
5	Un borrado por rango de líneas se llevó la declaración de una variable y el botón de enviar dejó de hacer nada, **sin error visible**	usando el producto
6	Al permitir ver el formulario sin haber enviado nada, siete usos que daban la respuesta por segura tumbaron la pantalla entera	usando el producto
7	La pestaña del formulario se pintaba sin estilos, y «limpiar» no la borraba	usando el producto
8	Un borrado de bloque cortado por el primer cierre dejó nueve líneas huérfanas y tres variables sin uso	una comprobación posterior del propio asistente

El reparto es el hallazgo. Seis de los ocho los encontró una persona usando el producto. Uno lo cazó la construcción de la imagen. Uno, una comprobación que el propio asistente hizo después de dar el cambio por terminado. Y ninguno lo cazó el método con el que el asistente verificó sus cambios en todo ese tramo, contar cadenas de texto dentro del paquete construido. Ese método confirma que algo está: no dice que se lea bien ni que funcione.

Dos de los ocho son el mismo error repetido el mismo día: borrar código por rango de líneas sin comprobar qué quedaba al lado. La primera vez se llevó por delante la declaración de una variable. La segunda, el cierre de un condicional anidado. De ahí sale una regla concreta, que vale más que la buena intención de «revisar mejor»: los bloques se delimitan por contenido, no por el primer cierre que aparezca, y después se comprueba qué ha quedado huérfano.

Lo que se saca de aquí es que la velocidad del asistente para aplicar un cambio no va acompañada de la misma velocidad para comprobar sus efectos laterales, y que sus verificaciones son sistemáticamente ciegas a lo que ve una persona en dos segundos. Por eso las capturas de producto las toma una persona y por eso el reparto del equipo pone las pruebas en manos humanas: no es desconfianza, es que los dos métodos fallan en sitios distintos.

Qué aportó el asistente, tramo a tramo

El modelo de trabajo lo fijó el grupo: se le ceden al asistente los mandos de construcción, y lo que no puede hacer se reparte a personas y queda escrito, probar el producto en marcha, revocar claves, autorizar lo irreversible. El recuento de la última columna sale del diario, contando sus entradas una a una.

Tramo	Qué construyó el asistente	Qué fue decisión o verificación humana	Registro

Reconciliación e historial	Ensayo y ejecución de la reescritura del historial y del empuje forzado de las siete ramas	**La revocación de las claves**, repartidas en tres cuentas: no es algo que el asistente pueda hacer. El visto bueno al empuje forzado	9 entradas · 4,6 h
Exposición y endurecimiento	Retirada de los puertos directos, orígenes permitidos, canal en tiempo real autenticado, topes de sesión y endurecimiento del servidor	**Probar la web en producción**: nueve pruebas, tres hallazgos y una corrección al asistente	19 entradas · 9,0 h
Identidad y aislamiento	Alta y acceso por usuario, el guardián de los espacios de datos, las cuatro fugas y la quinta que introdujo la propia corrección	**El modelo de alta**: con código de invitación y no abierto. Y la verificación en la instalación pública	21 entradas · 15,9 h
IA, pulido y cierre	RF-08 de punta a punta, quince defectos del rastreador y dieciséis del banco de peticiones, el manual de uso, el cierre del desarrollo y esta memoria	**Encontrar la mayoría de los defectos usando el producto**, las capturas de pantalla, la clave de la API y las decisiones de **qué no se toca** al cerrar el desarrollo	118 entradas · 47,1 h

Generado frente a verificado: la regla, y lo que cuesta sostenerla

- **Todo el código añadido o modificado en esta práctica lo escribió el asistente**, el de la práctica anterior lo escribió el grupo, y nada se dio por bueno por haberlo escrito: lo validan sondas con valor esperado y control negativo, pruebas en el entorno de trabajo, y la verificación en producción, que la hace una persona.
- **Las cifras de esta memoria salen de medidas con herramienta, y donde no se ha medido, se dice.** El apartado 6 es el ejemplo: da el número de avisos del auditor de dependencias y declara en la misma línea que no se ha medido cuántos son alcanzables.
- **Las dos formas en que este asistente se equivoca, nombradas porque se repiten:** un medidor que no sabría cantar otra cosa (un cero sobre un PDF cuyo texto va comprimido) y un documento que envejece en horas (un pendiente describe lo que alguien entendió al escribirlo, no lo que el código hace ahora, y fiarse de él produjo tres pendientes caducados en una sola respuesta).
- **Y la limitación de fondo:** el asistente no recuerda las sesiones anteriores. Lo que no quede escrito en el diario o en esta memoria no existe para la sesión siguiente. De ahí que el diario se escriba en el momento: es la única forma de que el motivo de una decisión sobreviva a quien la tomó.

SECCIÓN 12

Anexos

Manual de despliegue ampliado, cómo se entrega el acceso para la corrección, glosario y referencias.

Manual de despliegue

Esto es el procedimiento de actualización de la instalación pública. La instalación desde cero en una máquina nueva no se repite aquí: está en el README del repositorio, paso a paso y probada de punta a punta.

El despliegue es manual y deliberadamente aburrido: la instalación pública solo recibe versiones, nunca arreglos. Está prohibido editar ficheros en ella, fue la causa de que al principio de la práctica lo desplegado no estuviera en ninguna rama, y por eso la actualización se hace con traer y reponer a la fuerza el estado de la rama principal, no con una fusión: si alguien hubiera editado algo allí, el siguiente despliegue se lo lleva. Es intencionado.

Orden de los pasos, y por qué este y no otro:

1. Punto de retorno antes de tocar nada: etiquetar las imágenes en servicio y copiar la configuración. No cambia nada y es lo que permite volver en un minuto.
2. Fusionar la rama de trabajo en la principal mediante solicitud de incorporación. La rama principal está protegida y la solicitud sale bloqueada por la regla de revisión. La desbloquea quien administra.
3. Desplegar: traer y reponer a la fuerza, y levantar solo los servicios que el cambio toca.
4. Verificar con sondas, nunca por impresión visual.
5. Si falla, volver al punto 1.
6. Etiquetar la versión entregada.

Qué servicios se levantan, y esto se mide, no se supone. Se listan los ficheros que cambian entre las dos ramas y se agrupan por carpeta. En el último despliegue: tres servicios se reconstruyeron (interfaz, API y módulo de IA), uno se recreó sin reconstruir, solo ganaba una línea de red, y su imagen pesa 2,4 GB, y dos no se tocaron porque ni su imagen ni su configuración cambiaban. Dos lecciones que costaron tiempo y conviene no reaprender:

- **Nombrar un servicio no basta para recrearlo.** Levantar con reconstrucción un servicio cuya imagen se descarga y cuyo bloque no cambió no lo recrea: el despliegue sale «en verde» sin aplicar la configuración nueva. Para eso hace falta pedir la recreación explícitamente.
- **Quitar un servicio del fichero de composición no para su contenedor:** queda huérfano y sigue corriendo. Para que desaparezca de verdad hay que pedir que se retiren los huérfanos, y eso es parar y eliminar un contenedor, así que lo decide y lo lanza una persona, no el guion.

Verificación por sondas. Cada sonda tiene un valor esperado medido antes en el entorno de trabajo, y la lista incluye un control negativo: un marcador que debe dar cero. Sin ese control, una sonda que no encuentra nada es indistinguible de una sonda mal apuntada. Dos de las sondas de la primera versión estaban mal dirigidas, preguntaban a la pieza equivocada, y se cazaron probándolas, que es para lo que se prueban.

Avisos que no son opcionales:

- La reposición a la fuerza no toca el fichero de entorno (está excluido del control de versiones): la instalación pública conserva su clave y su código de invitación.
- Las variables nuevas traen valor por omisión, así que no hay que tocar el entorno para desplegar. Pero el tope de tokens por omisión se queda corto con el modelo actual: la respuesta se trunca y el análisis sale como «no analizado» (RNF-06). Y subirlo no es la salida: el tope por omisión está justo por debajo del límite que el modelo admite sin entregar la respuesta por partes, y el cliente no la pide por partes en ninguna llamada, así que al subirlo la consulta falla con un error explícito. Lo que queda, mientras no se añada esa entrega por partes, es convivir con el truncado: el análisis sale no analizado con su motivo, que es justo lo que RNF-06 pide. El propio producto recomienda subirlo en su mensaje de error, y ese consejo también hay que corregirlo.
- La reconstrucción de la interfaz es la pesada: si queda poca memoria libre, conviene saberlo antes.

Usuarios de prueba y acceso para la corrección

El producto ya no tiene una contraseña compartida: desde el cambio de acceso cada persona se da de alta ella misma con un código de invitación y elige su contraseña, que nadie más conoce. Eso cambia cómo se entrega el acceso para corregir:

- **En este repositorio no hay ninguna credencial**, ni de prueba ni ficticia. El repositorio es público: una contraseña escrita aquí sería una contraseña publicada, y la instalación a la que abre está en internet.
- **Las credenciales de corrección se entregan por el canal del máster**, junto a la dirección de la instalación. Incluyen lo necesario para entrar y, si se quiere crear una cuenta propia, el código de invitación.
- **Las cuentas que se usaron para probar** durante el desarrollo viven en el entorno de trabajo, no en la instalación pública, y se retiran antes de la entrega. Se nombraron con prefijos reconocibles por propósito, alta, rastreo, víctima y atacante, precisamente para poder localizarlas y borrarlas luego.
- **Qué se puede comprobar con una sola cuenta:** todo el recorrido de auditoría. Para comprobar el **aislamiento** hacen falta dos, y así se verificó: once de once comprobaciones con dos cuentas distintas.

Glosario

- **Spider (rastreador):** recorre un sitio siguiendo enlaces y formularios para levantar su mapa.
- **Repeater:** banco de trabajo de una petición: se modifica y se reenvía tal como se ve en pantalla.
- **Intruder:** prueba muchos valores en un punto marcado de la petición (fuzzing) y señala las respuestas que se salen del patrón.
- **Proxy del lado servidor:** el producto hace la petición en lugar del navegador, de modo que no hace falta instalar ni configurar nada en el equipo de quien audita.
- **Fichero de autoconfiguración de proxy:** mecanismo de la primera versión para que el navegador enrutara solo. Se jubiló al rehacer el acceso, junto con sus rutas.
- **Token firmado (JWT):** credencial que emite el servidor y que el cliente presenta en cada petición. Aquí el algoritmo de firma está fijado en el servidor y no se lee del propio token.
- **Hash de contraseña:** huella irreversible. Lo que se guarda es la huella, nunca la contraseña.
- **Bus interno:** canal por el que los servicios se hablan sin exponerse a internet. Sustituye a que un módulo llame a la API como si fuera un cliente más.
- **Operación degradada:** seguir dando servicio con menos garantías cuando una pieza falla, y que se note. Un fallo silencioso que se disfraza de resultado correcto es peor que una caída.
- **Techo de gasto:** límite de llamadas al modelo por auditoría. Al agotarse, el resultado sale degradado y visible en lugar de silencioso.
- **Trampa de araña:** sitio que genera direcciones infinitas y hace que un rastreador no termine nunca.
- **Fuga de aislamiento:** que los datos de un usuario aparezcan en la sesión de otro.
- **Huella digital (fingerprint):** identificar la tecnología de un objetivo por sus respuestas.
- **Entorno de trabajo y instalación pública:** las dos instalaciones del producto. Todo se monta y se prueba en la primera. La segunda solo recibe versiones (apartado 4).
- **Contenedor huérfano:** el que sigue corriendo después de que su definición desapareciera del fichero de composición.

- **RF / RNF:** requisito funcional (lo que el producto hace) y no funcional (cómo lo hace: seguridad, rendimiento, despliegue).
- **STRIDE:** modelo de amenazas que recorre seis familias, suplantación, manipulación, repudio, divulgación, denegación de servicio y elevación de privilegio, para no olvidar ninguna (apartado 6).

Referencias

Fuentes del enunciado y de la práctica anterior (material del máster, no redistribuido en este repositorio): el enunciado de esta práctica, el de la práctica anterior y el informe que el grupo entregó entonces. De ahí salen la estructura obligatoria de trece apartados, la numeración de requisitos que la práctica anterior no tenía y el plan de mejoras cuyos cinco primeros puntos delimitan el alcance (apartado 3).

Dentro de este repositorio, y todas estas rutas existen, se comprobó al escribirlas: docs/informe/P3-memoria.typ (fuente única de esta memoria), docs/informe/diario.md (registro de trabajo con el motivo de cada decisión, las horas y lo que se descartó), docs/capturas/ (figuras, con el nombre atado al requisito que demuestran) y docs/evidencias/ (salidas de terminal guardadas como texto, no como foto, para que sean buscables y comparables).

Taxonomías y modelos usados: la clasificación de severidad que pide RF-08 se apoya en la taxonomía de OWASP, y el modelo de amenazas del apartado 6 sigue STRIDE.

Herramientas de medición citadas en la memoria, para que cualquiera pueda reproducir las cifras: el auditor de dependencias de cada ecosistema (el del gestor de paquetes de la interfaz y pip-audit para los módulos en Python), pdftotext para escanear el PDF entregable en busca de secretos, sobre el PDF crudo el texto va comprimido y la búsqueda daría cero siempre, y el compilador de documentos con el que se generan las dos salidas de esta memoria.

Material de trabajo del grupo que no vive en este repositorio (protocolo de trabajo, flujo de ramas, procedimiento de despliegue y planes de trabajo) y que se cita en la memoria por su contenido, no por su ruta, precisamente para no dejar referencias que un lector del repositorio no podría abrir.